

Content

Abstract	I
Declaration	II
Acknowledgments	III
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.4 Scope and Limitations	3
1.5 Report Organization	4
2 Literature Review	5
2.1 LLMs in Enterprise Applications	5
2.2 Multi-Agent Systems and Orchestration	5
2.3 Code Generation and Text-to-SQL	6
2.4 RAG	7
2.5 Tool-Augmented Language Models	8
2.6 Data Security and Local Deployment	8
3 Requirements Analysis	10
3.1 Banking Industry Context	10
3.2 Functional Requirements	10
3.3 Non-Functional Requirements	12
3.4 Use Case Analysis	14
4 System Architecture Design	18
4.1 Five-Layer Architecture Overview	18
4.2 Frontend Architecture	20
4.3 Backend Microservice Architecture	23
4.5 Data Layer Architecture	29
4.6 Technology Stack Rationale	31
5 Core Agent Implementation	34
5.1 Tool Agent Implementation	34
5.1.1 Architecture and AI Intent Routing	34
5.1.2 Meeting Room Management	36
5.1.3 Schedule Conflict Detection	38

5.1.4	Route Planning	41
5.2	Code Agent Implementation	45
5.2.1	Architecture Overview	45
5.2.2	Python LLM Inference Server Integration	46
5.2.3	Five-Layer SQL Whitelist Validation	47
5.2.4	Metadata Caching and Execution	52
5.3	RAG Agent Implementation	55
5.4	Copilot — A Unified Intelligent Dispatcher Above the Three Agents	58
5.4.1	Design Rationale	58
5.4.2	Architecture Overview	58
5.4.3	Three-Tier Intent Classification	59
5.4.4	Frontend Dispatch and User Experience	60
5.4.5	Multi-Turn Clarification	61
5.4.6	Cross-Page Communication	61
5.4.7	Second-Level Intent Classification in the Tool Agent	61
5.4.8	AI Provider Configuration Unification	62
6	System Integration and Deployment	60
6.1	User Center and RBAC	60
6.2	Task Center and Orchestration	61
6.3	WebSocket Real-Time Communication	62
6.4	Containerized Deployment	63
7	Discussion	65
7.1	Challenges and Resolutions	65
7.2	Comparison with Existing Solutions	66
7.3	Project Limitations	66
8	Conclusion and Future Work	67
8.1	Summary of Achievements	67
8.2	Future Work	67
	References	69
	Appendix B: API Documentation (Selected Endpoints)	73
	Appendix C: User Manual (Summary)	76
	Declaration of Individual Contributions	78
	Declaration of Individual Contributions	78

Abstract

Large language models (LLMs) are developing rapidly, opening up new paths for enterprise digital transformation. However, applying them in regulated domains such as banking presents significant challenges. For example, generated code may be incorrect, domain knowledge can be outdated or unverifiable, enterprise tools may be invoked unreliably, and reliance on external APIs introduces data sovereignty risks. To solve these issues, this paper presents BankAgent, a multi-agent AI platform designed for banking digital transformation.

The platform integrates three agents: a Code Agent that converts natural language into SQL through a whitelist-validated Python inference pipeline, a Retrieval-Augmented Generation (RAG) Agent that retrieves policy knowledge using Milvus vector indexing, a configurable embedding worker, and an OpenAI-compatible LLM generation endpoint, and a Tool Agent that handles everyday business tasks such as meeting room booking, schedule conflict detection, and route planning. A unified Copilot dispatcher sits above all three agents, classifying user intent from a single natural language interface and routing each request to the appropriate agent.

The platform enforces role-based access control (RBAC) with tiered security permissions and department-level data isolation. Its five-layer microservice architecture spans a Vue 3 frontend, Spring Cloud Gateway, Spring Boot business and AI services, and a multi-engine data layer, all deployed via Docker Compose.

Declaration

This report is our own original work. We completed it independently. It has not been submitted before for academic qualification requirements at the University of Hong Kong or any other institution. All references to other people's work have been clearly cited and listed in the References section. We have read and followed the University's guidelines on academic integrity. We understand that plagiarism means copying someone else's work without giving credit. We also understand that the University takes plagiarism seriously.

Acknowledgments

We would like to extend our sincere gratitude to our supervisor, Dr. Huang, for his guidance, encouragement, and insightful advice throughout this project. His expertise shaped the direction of our work, and his steady support made the entire experience more rewarding. We are also grateful to the School of Computing and Data Science at the University of Hong Kong for providing the resources and learning environment needed to complete this work. We are grateful to all the faculty members and teaching staff. Their lectures in the MSc in Computer Science programme gave us the theoretical and practical grounding that underpinned this project. We would also like to thank our classmates and peers who took part in discussions. Their feedback during the development and testing phases helped us notice edge cases we had missed. Finally, we thank our families for their patience, encouragement, and unwavering support throughout this journey.

1 Introduction

1.1 Background and Motivation

LLMs are built on transformer architectures. They are trained on large sets of text and have pushed artificial intelligence forward. These systems can understand almost every language, create runnable code from a text prompt, and give detailed answers about many field. This is why engineers see them as a key piece of enterprise digital change.

But using LLMs in business banking brings problems that go beyond just linking up the tech. First, SQL that is made by the model for database queries is not always reliable. The code often has mistakes in structure or meaning, especially when the database has a complex layout. Second, AI models have limited knowledge of bank-specific rules. The rules change often, and model weights learned during training cannot reflect updates in policies. Third, bank workers must integrate AI into their daily work, such as room booking, schedule checking, and route planning. However, present AI models are not always good at calling the right tool or reading the right input.

The most serious problem comes from the common industry practice. Many firms rely on outside LLM APIs, like those from big cloud providers. This brings up data safety issues and legal control problems. The Hong Kong Monetary Authority (HKMA) has strict rules about handling customer information. The rules suggest that institutions must keep control over data access and processing. Therefore, sending internal queries or customer data to an outside API for AI reasoning is not safe in terms of compliance. It is against the principle of data sovereignty.

The current work is driven by several linked problems. These are technical precision, keeping domain knowledge correct, stable tool calling, and data control. So, a single multi-agent AI system is clearly needed. This system must work on-site and take a defense-in-depth approach. It must combine secure code generation, local knowledge retrieval, and enterprise tool calling.

1.2 Problem Statement

The present work are facing four major challenges.

Code generation accuracy. Database queries are used in everyday bank work. For example, they are used to get customer data and to file reports required by rules. Natural-language interfaces can make data access much easier. But LLM-written SQL is not always safe. It must be run through careful checks. The model may add syntax faults, wrong column references, or even harmful operations. The system needs to check multiple levels before executing any SQL statement.

Knowledge retrieval. Answers to procedural and policy questions must be verifiable. They must be backed by citations. This is important for bank employees. Finance is a field where rules must be followed and there is zero tolerance for errors. RAG methods are used to fix LLM issues with old or made-up information. But bank documents have security levels. The system must filter retrieval results to match the user's clearance level. Documents must not leak through answers.

Enterprise Tool Integration. Daily bank work uses many tools, including room booking, calendar syncing, and route planning. An AI system must correctly understand user requests. It must find the right intent, extract the right parameters, and run the matching tool. The system should also use large language model routing to pick the best handler for each request.

Data Sovereignty And Access Control. Beyond the risks from outside APIs, banking settings need strict access levels. Different departments handle documents with different security clearances. The rule of least privilege must be enforced at every layer. Small permission models are often too simple for an organization that has many departments and levels of leadership.

1.3 Project Objectives

This work presents BankAgent, a multi-agent AI platform built on on-premise infrastructure and designed for banking operations. The platform delivers three AI agents under a unified Copilot dispatcher, supported by five platform services and one-command containerized deployment. Table 1 summarizes the project objectives.

Table 1: Project Objectives by Platform Layer

Category	Component	Objective
AI Agents	Code Agent	Safely translate natural language questions into executable SQL queries.
	RAG Agent	Retrieve policy knowledge from Milvus-indexed documents with verifiable citations and clearance based permission filtering.
	Tool Agent	Automate business tasks such as room booking, schedule conflict detection, and route planning through natural language.
Intelligent Dispatch	Copilot	Classify user intent from a single interface and dispatch requests to the appropriate agent.
Platform Services	Auth & RBAC	Enforce JWT authentication with three role tiers and department scoped data access.
	API Gateway	Provide a unified entry point with JWT filtering, rate limiting, and session enforcement.
	Notification System	Manage multi type messages with a five state approval workflow for sensitive operations.
	Document Management	Store and serve documents with tiered security clearance and HITL escalation.
	Task Center	Orchestrate asynchronous agent execution with retry logic and real time progress streaming.
DevOps	Containerized Deploy	Start all services with a single Docker Compose command.

1.4 Scope and Limitations

This project covers the design, implementation, and evaluation of the multi-agent platform described above. The intended users are banking personnel. The platform provides three main functions. It generates and checks SQL queries. It does knowledge retrieval with citations. It calls business tools. The deployment model is local. It uses Docker Compose for container management. The security model includes JWT authentication and RBAC.

But there are still several limits. First, the Code Agent supports only MySQL SQL and relational table structures. It does not work with other database systems or NoSQL interfaces. Second, the RAG Agent’s retrieval quality depends on the selected embedding

model and the coverage of indexed documents. Third, the Tool Agent only supports three business tools at present. Its scope is defined by the current set of enterprise APIs. Fourth, the platform is built for a single deployment and does not yet support Kubernetes or multi-node scaling.

1.5 Report Organization

The rest of this report is arranged like this. Chapter 2 looks at past work on LLM use in business settings, systems with many agents, turning text into SQL, RAG methods, language models with added tools, and on-site use and data safety. Chapter 3 explains what banks need, functional and non-functional requirements, and user case studies. Chapter 4 shows the five-layer system design, the frontend and backend layouts, data layer construction, and the reasons for choosing each technology. Chapter 5 explains in detail how the three main agents are built. Chapter 6 covers the process of putting the system parts together, including user management, task planning, live communication, and deploying with containers. Chapter 7 talks about the project's difficulties, how this work compares to other solutions, and where it falls short. Chapter 8 sums up the work that was finished and suggests what should be looked at next along with future work.

2 Literature Review

2.1 LLMs in Enterprise Applications

Transformer-based LLMs have changed AI research and its uses. Vaswani et al. (2017)[9] introduced the transformer architecture. Since then, this field gain a rapid progress, large models, like GPT-4, Claude, DeepSeek has come out. These models performs very well in understanding natural languages and inference tasks. Its core relies on self attention mechanism, which can capture semantic associations within the scope of long texts. Through this, coherent text will be generated.

In some specialized organizations, LLMs are used to automatically manage knowledge intensive works, including report writing, document abstract generating, programming code making, specialized field questions answering and complex work flow managing. But if the company use ready-made models, they may face lots of problems. Bender et al. (2021) [11] call these models "Random Parrot". They believe that models can only learn statistical laws from training data, with no really understanding ability. So they may output seemingly reasonable answers, which is wrong actually. In regulated industries like bank, this is worrying because it may led to legal penalties and economic losses.

Financial services industry has a strict attitude on using AI. Banks must meet the regulation commands. They need to standardize data confidentiality, transaction audit, and explanatory automate decision. The Basel Committee on Banking Supervision has issued guidelines on AI in the financial industry, requiring transparency, resilience testing, and manual supervision of model outputs.

2.2 Multi-Agent Systems and Orchestration

Multi agent systems (MAS) originated from research focus on distributed artificial intelligence. Early systems used rule-based agents, which had fixed interaction rules. As LLMs are becoming more and more popular, a new architecture design based on intelligent agents has emerged. Under this architecture, each agent relies on language models to complete reasoning, planning, and interactive communication between agents.

Wu et al. (2023) proposed AutoGen [4]. This platform allows different LLM-driven agents to communicate, and work together to solve complex questions. AutoGen has a

dynamic conversation mechanism, which can allocate each agent a role, so that subtasks can be transferred and peer review can be used to optimize outputs. But this frame is unpredictable and its output may not be reliable, which is unbearable in commercial scenarios. On the opposite, Hong et al. (2024)[5] proposed MetaGPT. This frame embeds Standard Operating Procedures (SOPs) into a multi-agent collaboration system, drawing inspiration from the working mode of factory assembly lines. Virtual roles such as product managers, system architects, development engineers, and testers are set up, and agents collaborate through the transfer of standardized intermediate results.

These frameworks show promising results in research settings. However, there's still a gap between flexible, role-based multi-agent systems and production systems in banking environments. For process standardization, systems must present deterministic behaviors and tight security controls.

2.3 Code Generation and Text-to-SQL

Converting natural language questions into executable SQL statements has been a major challenge in the field of natural language processing for over 20 years. The early solution adopted a rule-based approach, relying on manually constructed grammar rules and domain specific knowledge systems. This type of method is effective in narrow domain scenarios, but it is difficult to adapt to diverse database structures.

The emergence of the Spider dataset is an important development in this field. Yu et al. (2018) [1] constructed this dataset. Spider is a cross domain large-scale test set that includes 10181 query examples from 138 domains and 200 databases. Its core is used to test the model's ability to handle unfamiliar database architectures. This demand is highly compatible with the banking industry scenario: financial databases will be frequently updated due to new product launches, regulatory reporting requirements, and market data changes.

2.4 RAG

RAG fixed a weakness in LLM. LLM relies on the static knowledge stored in its parameters. This knowledge often leads to factual errors. Lewis et al. (2020) [8] first described RAG. The RAG design has added a step of obtaining information from external sources.

This step retrieves relevant paragraphs from the knowledge base. It provides model evidence for generating answers.

A typical RAG workflow consists of three stages. The first step is index construction: source data is segmented into chunks and then converted into a supported representation (embeddings) using an encoder. Next, LLM generates the final output. This design distinguishes language processing capabilities from actual data storage, enabling generated responses to leverage an organization's specific, real-time data.

RAG technology is highly valuable in the banking sector, as regulatory rules, internal policies, and product documentation are typically stored in vast text repositories. Furthermore, this content is frequently updated, making it impossible to integrate directly into model weights. Pipitone and Alami (2024) introduced LegalBench-RAG [2], a benchmark specifically designed to evaluate RAG systems in the legal and financial domains; this benchmark focuses on clause-level precision rather than document-level recall.

2.5 Tool-Augmented Model

LLMs have inherent limitations that prevent them from directly interacting with external systems or data stores. Tool-augmented language models effectively solve this problem by equipping LLMs with the capability to call APIs, query databases, and perform computational tasks. Yao et al. (2023) [10] introduced the ReAct paradigm, which combines "Reasoning" and "Acting", demonstrates that integrating reasoning steps with actions enables LLMs to use external tools for tasks involving real-time data, mathematical calculations, and environmental interactions.

In enterprise applications, the scope of tool usage not only consists of standard web searches, but also specialized tools tailored to specific organizational needs, such as scheduling, resource booking, internal data querying, and process automation. The reliability of tool invocation is a critical performance metric, particularly regarding user intent understanding, precise parameter extraction, and error recovery.

2.6 Data Security and Local Deployment

Such tool invocation, however, is often realised through external cloud APIs, which raises additional security concerns. Security is core consideration of enterprises adopting AI

systems. Currently, a common practice involves accessing LLMs via cloud APIs. That is, external endpoints accept and process users' queries and context. This method obeys the data governance policies enforced in regulated sectors. As Karras et al. (2025)[6] noted, sensitive information in prompt payloads may be logged by providers, repurposed for model retraining, or leaked through inversion techniques.

In the banking sector, the Hong Kong Monetary Authority (HKMA) requires the implementation of appropriate safeguards to protect customer information, and also requires that relevant institutions retain control over data access and processing. Similar requirements are found in the EU's General Data Protection Regulation (GDPR), the Personal Information Protection Law (PIPL) of mainland China, and banking security regulations in other jurisdictions. Together, these regulatory frameworks require that confidential financial data be processed in controlled environments that provide full auditability, which is a requirement that cannot be met when data is sent to or processed by third-party services.

On-premises deployment faces a significant computational cost challenge in LLM inference, which Kwon et al. (2023) [7] addressed with the PagedAttention mechanism and the vLLM framework. This solution, inspired by operating system virtual memory management, employs specific memory management strategies to effectively alleviate GPU memory bottlenecks.

Karras et al. (2024) [6] provided a comprehensive review of LLM applications in cybersecurity contexts in the big data era. Their analysis synthesized findings from over 100 research papers covering vulnerability detection, incident response, threat intelligence, and secure code generation. Basically, they found that while LLMs show promise in automating many cybersecurity tasks—such as identifying vulnerabilities in source code and generating security patches—current models still struggle with problems like contextual understanding and high false-positive rates in real-world enterprise settings. For this project, these findings reinforce how important it is to adopt a defense-in-depth security architecture instead of relying on LLM reasoning alone.

3 Requirements Analysis

3.1 Banking Industry Context

The system requirements are derived from banking operations. Banking IT environments share four characteristics that drive the platform's requirements. (i) Large relational databases store client profiles, transaction histories, product catalogs, and compliance records. (ii) Policy and procedure repositories span multiple departments, each document carrying a security classification and subject to frequent regulatory updates. (iii) Administrative workflows such as room booking, calendar coordination, travel planning consume substantial staff hours and remain largely manual. (iv) Security and compliance obligations, including those mandated by the HKMA Supervisory Policy Manual (as discussed in Section 2.6), demand role-based access control, department-scoped data isolation, and auditable escalation processes.

Stakeholder analysis identifies five user groups. Business analysts need to query databases without deep SQL knowledge. Compliance officers need to quickly find relevant regulatory clauses with verifiable references. Administrative staff coordinate meetings and schedules across the organization. Departmental administrators manage team membership, document access rights, and approval workflows in their units. System administrators handle platform configuration, manage user permissions, and allocate resources.

3.2 Functional Requirements

The platform's functional requirements are organized by domain. Table 2 summarizes each requirement with its rationale and priority.

Table 2: Functional Requirements

ID	Requirement	Description
FR1	User Authentication & RBAC	Email-format registration with hashed passwords and stateless JWT authentication. Three roles (Admin, Department Admin, End-User) with fine-grained permissions enforced at the gateway level.
FR2	Department Management	Hierarchical department structure with six banking units. Department administrators can manage membership and assign clearance levels (Public, Internal, Confidential).
FR3	Document Management with Security Clearance	Documents stored with department ownership and security level. Access granted only when the user's department and clearance meet the document's requirements. Inaccessible documents trigger a human-in-the-loop escalation pathway.
FR4	Notification System	Multi-type notifications (chat, access request, SQL audit alert) with a five-state state machine. Unread badge and role-specific views provided on the frontend.
FR5	Code Agent — NL2SQL	Natural language to SQL translation with security validation before execution. Schema metadata cached to improve generation accuracy. Human-in-the-loop mode supported: generate, review, then execute.
FR6	RAG Agent — Policy Q&A	Natural language questions answered from organizational documents with verifiable source citations. Retrieval uses Milvus-indexed chunks, configurable embeddings, permission filtering, and audit records for retrieved and blocked documents.
FR7	Tool Agent — Task Automation	Unified natural language interface for meeting room booking, schedule conflict detection, and route planning. Intent recognized automatically and routed to the appropriate handler.
FR8	Admin Resource Management	Role-restricted CRUD endpoints for meeting rooms and schedules. Validation rules enforce required fields, positive capacity, and chronological time ranges.

3.3 Non-Functional Requirements

Table 3 summarizes the non-functional requirements that govern system quality attributes.

Table 3: Non-Functional Requirements

ID	Requirement	Description
NFR1	Data Sovereignty	All core relational storage, vector storage, document indexing, and access-control checks run within the on-premise environment. Administrators can configure whether AI inference and embedding use local models or external APIs. When external APIs are used, only the minimum permission-filtered payload is transmitted.
NFR2	Performance	SQL generation must complete within 3 seconds. Meeting-room and schedule queries must respond within 500 ms. Route planning must return within 2 seconds. RAG vector retrieval should complete within 1 second after the index is ready, while full answer generation depends on the configured LLM endpoint. WebSocket progress updates must be delivered within 200 ms. Initial page load must not exceed 3 seconds.
NFR3	Security	All traffic must use HTTPS (WSS for WebSocket). Passwords must be hashed with BCrypt. Authentication must be stateless via signed JWT tokens with configurable expiry. Access must be enforced at the method level. SQL injection must be prevented through parameterized queries. Data deletion must be logical rather than physical for auditability.
NFR4	Reliability	The platform must achieve 99.5% uptime during business hours. External service failures must trigger graceful degradation using mock fallbacks or permission-safe retrieval context rather than hard failures. RAG readiness checks expose Milvus, embedding, and LLM availability. All task failures must be logged with user context for post-incident analysis.
NFR5	Maintainability	Services must be independently deployable with clearly separated responsibilities. All APIs must return a consistent response wrapper for uniform error handling. Database operations must use type-safe abstractions with SQL logging for debugging.
NFR6	Deployability	The entire platform must start with a single command. Database schema, seed data, and RAG metadata tables must be initialized automatically. Service dependencies must be resolved through health-check probes before dependent containers start.

3.4 Use Case Analysis

The platform serves three human actor roles and interfaces with two external systems, summarized in Table 4.

Table 4: System Actors

Actor	Type	Capabilities
ROLE_ADMIN	Human	Full system access: manage users, roles, departments, clearance levels; configure AI providers and session policies; CRUD meeting rooms and schedules; view all tasks.
ROLE_DEPT_ADMIN	Human	Department-scoped access: manage membership and clearance levels within own department; create, edit, and delete department documents; review and approve document access escalation requests.
ROLE_USER	Human	Basic access: submit NL2SQL, RAG, and tool tasks through the Copilot or dedicated pages; query own task history; view and request access to documents within own department and clearance level.
AI Provider	External system	Receives natural language prompts for intent classification, SQL generation, and RAG answer generation. Administrators may configure local or cloud providers.
Amap API	External system	Converts addresses to coordinates (geocoding) and computes driving routes for the Tool Agent’s route planning capability.

The five core use cases spanning these actors are summarized in Table 5.

Table 5: Use Cases

ID	Actor	Goal	Agent	Main Flow
UC1	Business analyst	Query database without writing SQL	Code Agent	User enters a natural language question; LLM generates SQL; whitelist validation runs; query executes; tabular result returned.
UC2	Compliance officer	Find policy clauses with verifiable citations	RAG Agent	User asks a regulatory question; query embedded; top-k chunks retrieved and filtered by department and clearance; LLM generates answer with source citations.
UC3	Staff member	Book a meeting room via natural language	Tool Agent	User describes requirements; intent classified; parameters extracted; available rooms queried; options displayed.
UC4	Staff member	Detect schedule conflicts	Tool Agent	User specifies attendees and time range; each attendee's calendar checked; conflict report returned with alternatives.
UC5	Staff member	Access a document above own clearance	Notification System	Access denied; user submits escalation request; department admin reviews and approves; temporary access granted with audit trail.

Figure 1 illustrates the Tool Agent use cases in UML notation, showing the interaction between the three human roles, two external systems, and the three tool capabilities.

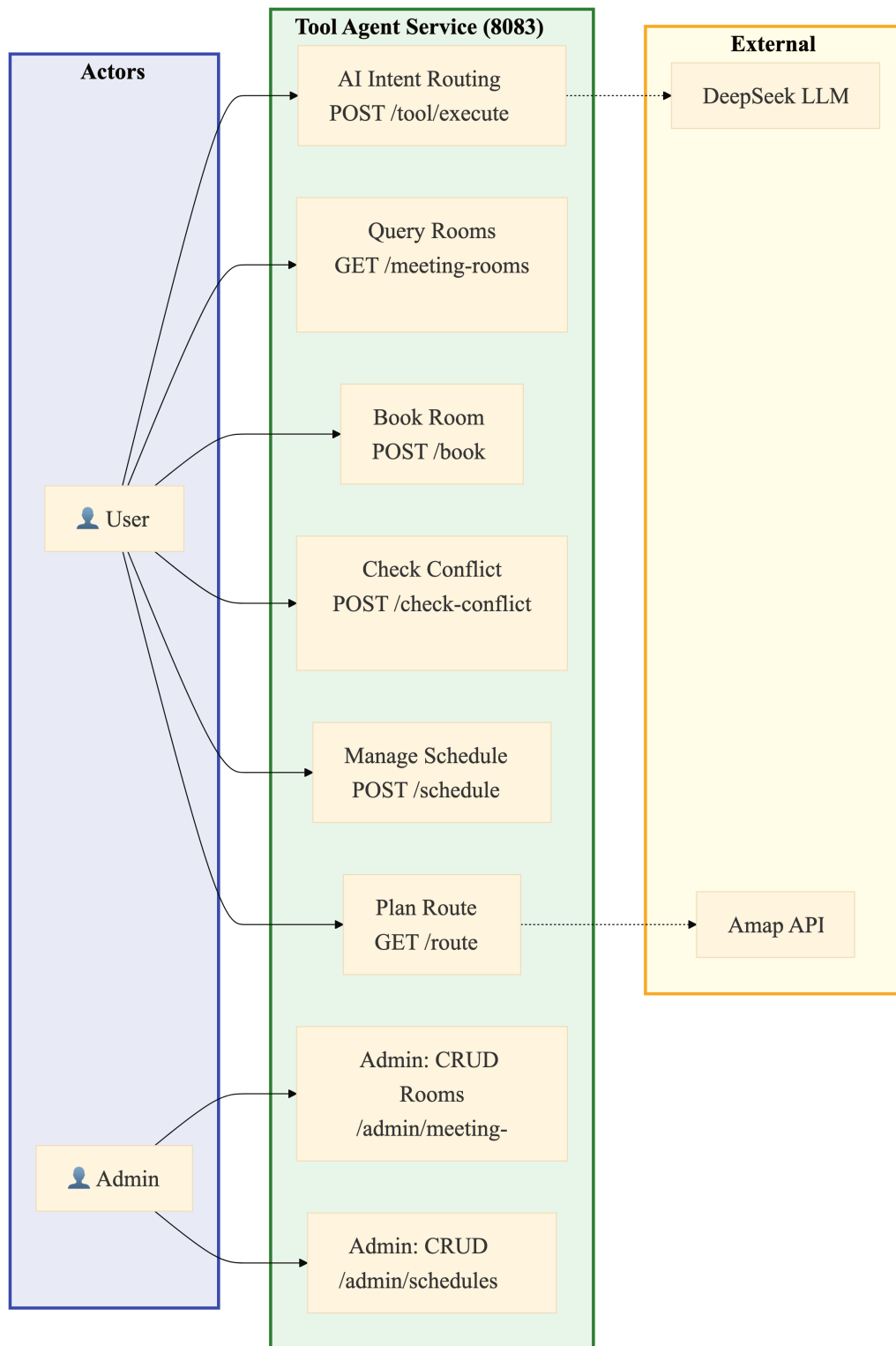


Figure 1: Tool Agent Use Case Diagram — Meeting Room, Schedule Conflict, and Route Planning

4 System Architecture Design

4.1 Five-Layer Architecture Overview

This platform adopts a five-layer architecture with high cohesion and low coupling, implementing one-way dependencies between layers: data is transmitted downwards via RESTful APIs, and long-lived connections are established upwards via WebSockets to proactively push events. This meets the requirements of banking systems for “modularity,” “testability,” and “independent scalability.”

The five-layer structure is shown in Figure 2.

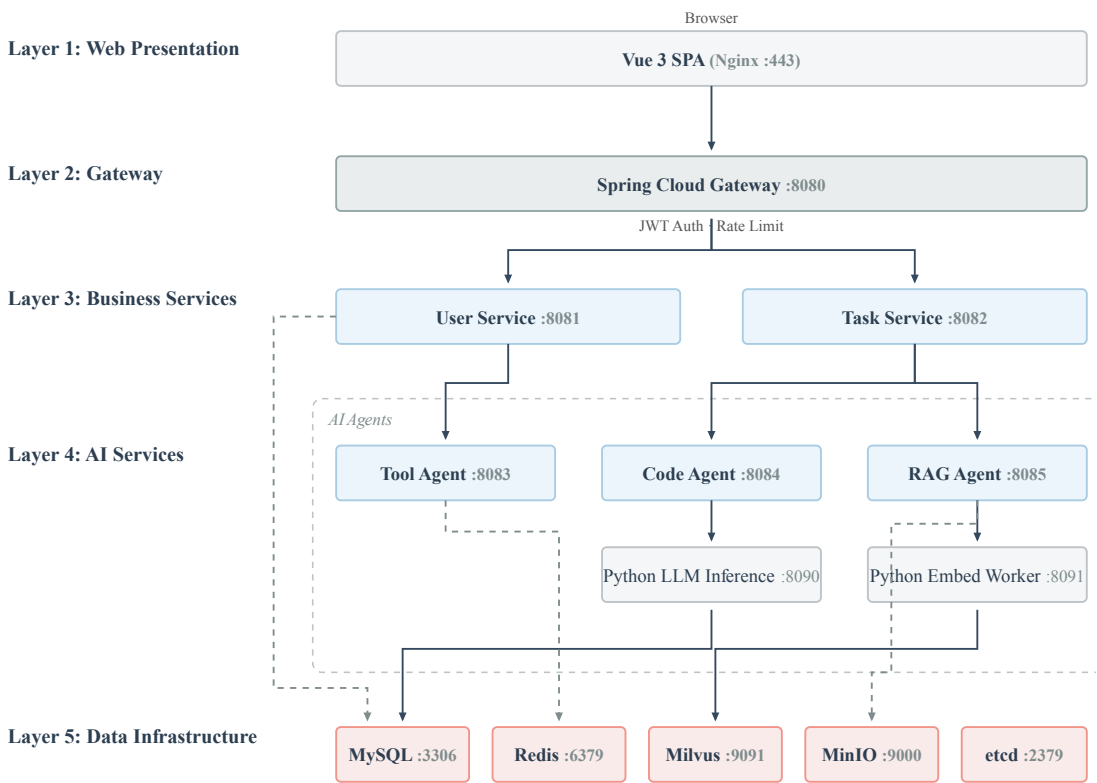


Figure 2: Five-Layer Architecture — All Services with Internal Ports

The five layers, from top to bottom, are as follows.

Layer 1 — Web Presentation Layer. This is the front end of the architecture, built with Vue 3 SPA, responsible for user interaction, result display (tables, maps, and charts), and real-time progress notifications via WebSocket. It supports three languages: Simplified Chinese, Traditional Chinese, and English.

Layer 2 — Gateway Layer. As the system’s sole public entry point, the gateway handles request routing and identity verification. Every request not on the whitelist must carry a valid JWT. Once verified, the user’s identity, roles, and permissions are extracted and forwarded to downstream services through custom headers.

Layer 3 — Business Service Layer. This layer contains user services and the task center. The user service manages registration, login, JWT issuance, role-based access control, notifications with a multi-type state machine, and document storage with department-scoped security clearance. The task center orchestrates cross-agent requests: it accepts tasks, queues them for asynchronous execution, tracks their progress, and streams status updates to the frontend via WebSocket.

Layer 4 — AI Service Layer. Three specialized agents handle the platform’s AI capabilities. The Code Agent converts natural language to SQL and validates it through a multi-layer security check before execution. The Tool Agent handles business tasks such as meeting booking, schedule conflict detection, and route planning, using LLM-based intent routing to classify and dispatch requests. The RAG Agent combines vector retrieval with LLM generation to answer policy questions with verifiable citations, filtered by the user’s department and clearance level. Sitting above all three, the Copilot dispatcher classifies user intent from a single natural language interface and routes each request to the appropriate agent. Two Python-based services support these agents: an LLM inference server for the Code Agent, and an embedding worker for the RAG Agent.

Layer 5 — Data Layer. This layer uses a mix of storage engines chosen for their specific roles. MySQL stores user accounts, permissions, departments, documents, meeting rooms, and other relational data. Redis caches hot data such as schema metadata and handles schedule conflict detection through sorted sets. Milvus stores document embeddings and serves semantic similarity queries for the RAG Agent. MinIO holds uploaded document files as objects. etcd coordinates Milvus cluster metadata.

The request lifecycle involves two distinct interaction patterns, illustrated in Figure 3. For short-running operations such as login, registration, and document CRUD, the browser sends a synchronous HTTP request through the Gateway directly to the relevant business service, which reads or writes to MySQL and returns a response. For AI agent tasks, the flow is asynchronous: the browser submits a task via the Copilot or an agent page, the

Task Service queues it for execution and routes it to the appropriate agent, and progress is streamed back to the browser via WebSocket. The Copilot adds a preliminary classification step — the user’s natural language query is sent to the Python Flask classifier to determine which agent should handle it before any task is created.

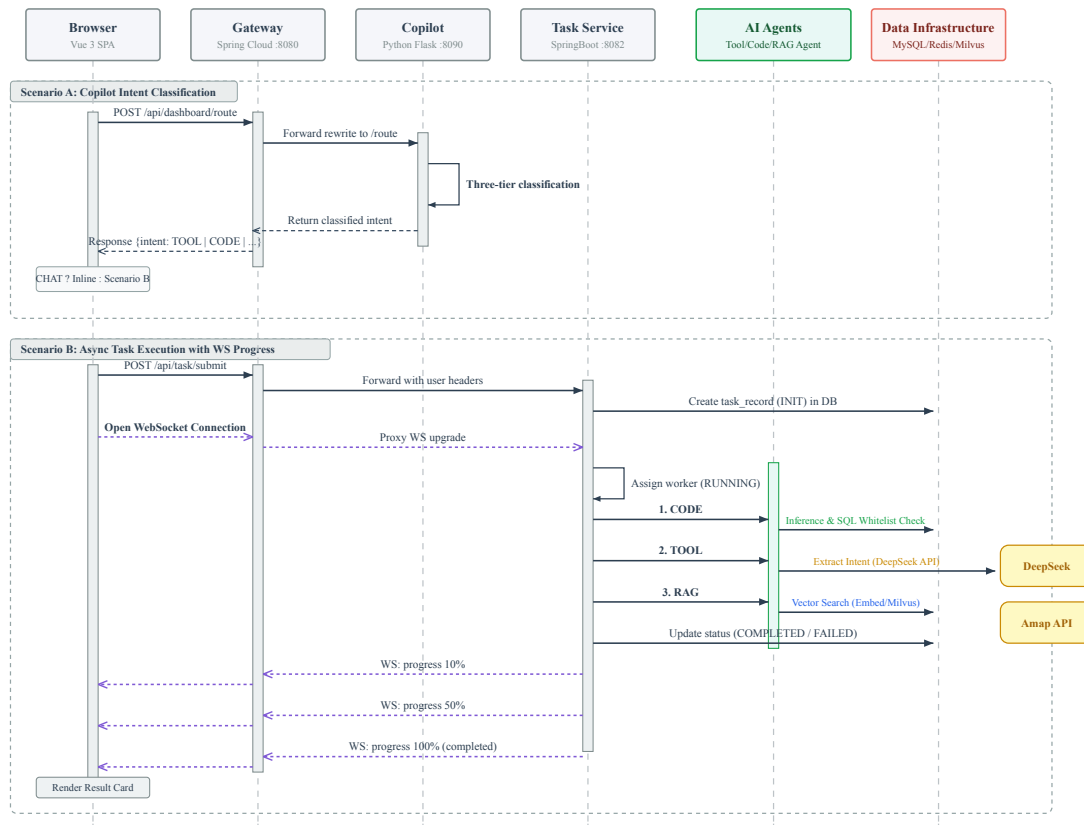


Figure 3: End-to-End Request Sequence — Copilot Intent Classification and Asynchronous Task Execution with WebSocket Progress Streaming

4.2 Frontend Architecture

The front end is built on Vue 3 with Composition API and TypeScript. TypeScript catches type errors at compile time. Composition API allows shared interaction logic to be reused across Tool, Code, RAG, and management pages.

(1) Design Language. The interface follows a minimalist design language: a grayscale color system, unified spacing, and a collapsible sidebar. The goal is a clean, quiet interface that keeps the user focused on the task.

(2) Role-Based View Control. The frontend adapts its interface based on the user’s role. The sidebar menu shows or hides entries depending on whether the user is an Admin, De-

partment Admin, or regular user. The management panel is only visible to administrators. The Copilot widget is globally available regardless of role. This means a regular user sees agent pages and basic settings, while an admin additionally sees user management, department management, and system configuration entries.

(3) Multi-Tab Navigation. A browser-style multi-tab bar sits at the top of the main layout. Users can open multiple agent pages, the management panel, and system settings in parallel tabs. When switching between tabs, each page preserves its full state — form inputs, filter selections, scroll positions, and in-progress results are not lost. A user can start a SQL query on the Code Agent page, switch to check a meeting room on the Tool Agent page, and return to the query result without interruption.

(4) Routing and Layout. Vue Router divides routes into public and authenticated groups. Navigation guards block unauthenticated access before route transitions. The authenticated layout has a collapsible sidebar, role-based menu visibility, and a five-layer navigation guard pipeline covering token recovery, authentication redirect, and role checks. Three languages are supported: Simplified Chinese, Traditional Chinese, and English. Unread notification counts are polled every ten seconds and shown as badges.

(5) State Management and Communication. Two Pinia stores manage frontend state: one for user authentication and permissions, the other for task lifecycle tracking. HTTP requests go through a shared Axios instance with JWT attachment, 401 auto-redirect, and unified error handling. All server responses follow a standard wrapper with code, message, and data fields. WebSocket connections use a singleton client with automatic reconnection, pushing real-time task progress to the UI without polling overhead.

Table 6 summarizes the frontend technology stack.

Table 6: Frontend Technology Stack Specifications

Component	Technology	Description
View Layer	Vue 3 + TypeScript	Composition API with type-safe development
State Management	Pinia	userStore (JWT, roles, permissions), taskStore (task lifecycle)
UI Components	Element Plus	Enterprise forms, tables, dialogs, and navigation
Visualization	ECharts 5	Dashboard charts and SQL result analytics
Build Tool	Vite	Fast HMR during development, Rollup-based production builds
HTTP Client	Axios	JWT auto-attach, 401 redirect, unified error handling
Real-Time	WebSocket	Singleton client with auto-reconnect and four event categories
Internationalization	Vue I18n	zh-CN, zh-TW, en locales
Map	Amap JS API v2.0	Dynamic SDK loading, route polyline, custom markers

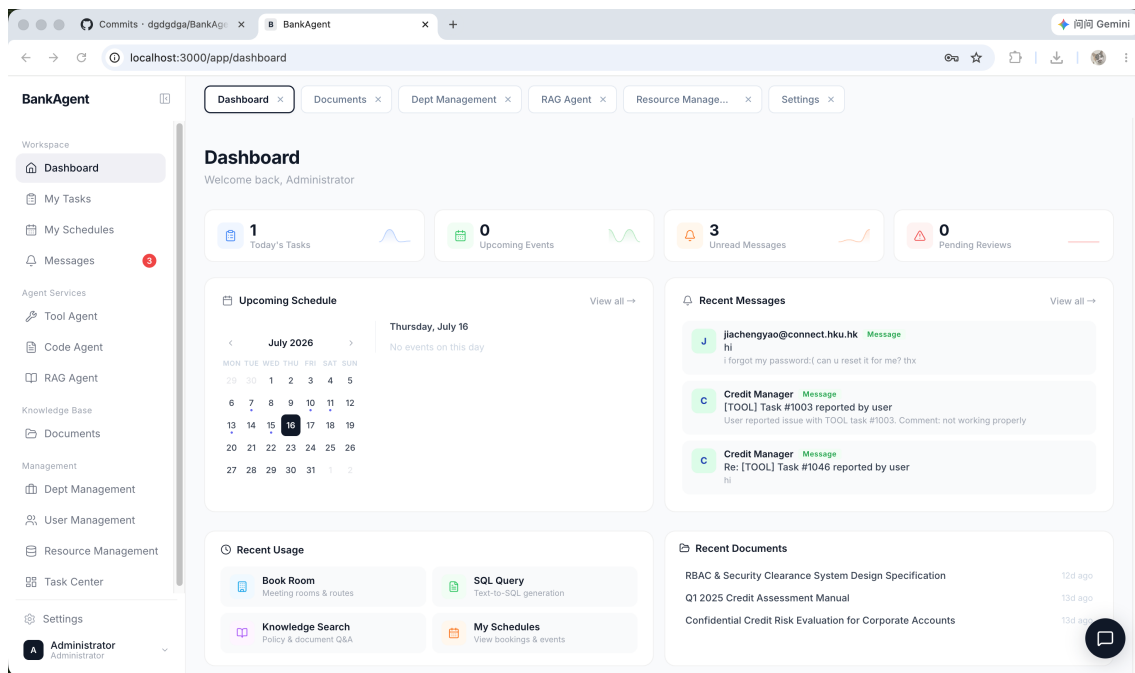


Figure 4: Platform Interface — Main Layout with Collapsible Sidebar, Multi-Tab Bar, and Copilot Floating Widget

4.3 Backend Microservice Architecture

The backend consists of independent Spring Boot microservices, each handling a specific business capability and communicating through the Gateway layer. This decomposition allows independent development, testing, deployment, and scaling of each service, while

the Gateway provides a single API entry point for the frontend.

(1) Gateway Service. The Gateway runs on port 8080, built on Spring Cloud Gateway. A global JWT filter validates tokens on all non-whitelisted paths and forwards user identity to downstream services through custom HTTP headers. Single-session enforcement invalidates old tokens when a new login occurs. IP-based rate limiting rejects requests exceeding the configured threshold. The Gateway is the only container exposed to the public internet; all other services communicate over the internal Docker network.

(2) User Service. The User Service runs on port 8081. It handles registration with email verification, dual-mode login via password or verification code, JWT issuance with roles and permissions embedded in claims, and three-tier RBAC with department-scoped data isolation. It manages documents stored in MinIO with clearance-level access control and HITL escalation for cross-clearance requests. A notification subsystem handles multi-type messages with a five-state state machine for approval workflows. The email verification subsystem uses the Resend HTTP API with three-tier rate limiting.

(3) Tool Agent Service. The Tool Agent runs on port 8083. It provides three business capabilities: meeting room booking, schedule conflict detection using Redis sorted sets, and route planning via the Amap API. A unified natural language entry point uses LLM intent routing to classify user requests and dispatch them to the appropriate handler. Dedicated admin endpoints support CRUD operations on rooms and schedules for authorized users.

(4) Code Agent Service. The Code Agent runs on port 8084. It converts natural language questions to SQL through a Python LLM inference proxy on port 8090. Generated SQL passes through a multi-layer whitelist validation before execution. Database schema metadata is cached for query generation accuracy. A human-in-the-loop mode lets users review SQL before it reaches the database.

(5) RAG Agent Service. The RAG Agent runs on port 8085. It ingests enterprise documents through parsing and structure-aware chunking, generates embeddings, and stores vectors in Milvus for semantic search. User questions are embedded, matched against the vector store, and answered with source citations. Retrieval is filtered by the user's department and clearance level. A Python embedding worker on port 8091 supports local model inference.

(6) Task Service. The Task Service runs on port 8082. It is the platform's asynchronous

execution backbone. It accepts tasks from the Copilot and agent pages, persists them, queues them for execution in a bounded thread pool, and routes each task to the appropriate agent. Failed tasks are retried up to three times with exponential backoff. Progress is pushed to the frontend in real time via WebSocket.

Table 7 summarizes the backend services.

Table 7: Backend Service Ecosystem and Technical Orchestration

Component	Technology	Description
Gateway	Spring Cloud Gateway	Dynamic routing, JWT validation, rate limiting, WebSocket proxy
User Service	Spring Boot 3.x + MyBatis-Plus	Authentication, RBAC, document management, notifications, email verification
Task Service	Spring Boot 3.x	Async task orchestration, retry with backoff, WebSocket progress streaming
Tool Agent	Spring Boot 3.x	Meeting booking, schedule detection, route planning, LLM intent routing
Code Agent	Spring Boot 3.x + Python FastAPI	NL2SQL generation, multi-layer SQL validation, metadata caching
RAG Agent	Spring Boot 3.x	Document Q&A, vector retrieval, source citation, clearance-filtered access
Security	Spring Security 6.x + JWT	Stateless authentication, RBAC with @PreAuthorize, BCrypt hashing
Caching	Redis	Session caching, rate limit counters, metadata cache, schedule zsets
LLM Inference	DeepSeek API (HTTPS)	Intent routing, SQL generation, RAG answer generation
Embedding	Python FastAPI (port 8091)	BGE-M3 embedding generation for document indexing and query
Document Parse	Apache Tika	Multi-format document parsing for RAG ingestion
Vector DB	Milvus	Semantic ANN search for RAG document retrieval
Object Storage	MinIO	Document file storage, Milvus backend
Email	Resend API	Verification code delivery, three-tier rate limiting
Maps	Amap API (HTTPS)	Geocoding, direction calculation, route polyline

4.4 Security Architecture

Data security is paramount in banking systems. The platform does not rely on a single line of defense, but rather incorporates defense in depth across four layers: gateway authentication, email verification with rate limiting, role-based access control with department-level data isolation, and transport-layer encryption.

4.4.1 Gateway Authentication

Figure 5 illustrates the complete authentication flow, from registration through email verification to JWT-based request authentication.

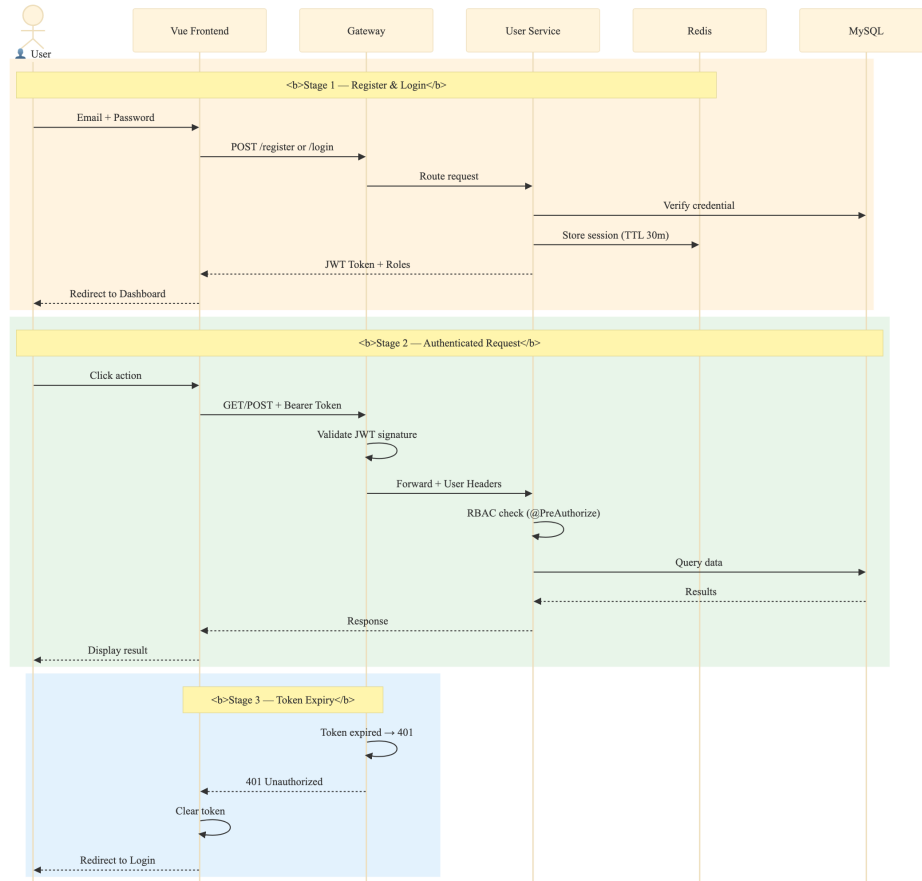


Figure 5: JWT Authentication and Email Verification Flow

The authentication model is stateless and token-based. On login, the user service validates credentials and issues a signed JWT containing the user's identity, roles, permissions, department ID, and clearance level. The token has a configurable expiry, and a Redis-based single-session mechanism ensures that a new login invalidates any previously issued token for the same user.

On every subsequent request, the frontend attaches the JWT as a Bearer Token via an Axios interceptor. The Gateway's global filter intercepts the request before routing and performs the following checks:

- **Whitelist bypass.** Login, registration, and verification code endpoints are publicly accessible without a token.

- **Signature verification.** The token is verified using HMAC-SHA256 with a secret key injected via environment variable.
- **Expiry check.** Expired tokens are rejected.
- **Session check.** The token is validated against the active session stored in Redis. If no matching session exists, the request is denied.
- **Identity propagation.** On success, user identity is extracted from the JWT claims and forwarded to downstream services through custom HTTP headers: `X-User-Name`, `X-User-Roles`, `X-User-Permissions`, `X-User-Dept-Id`, and `X-User-Security-Level`.

This header-based propagation means downstream services never hold the JWT secret. They trust the headers because the Gateway is the only entry point and cannot be bypassed from within the internal Docker network.

Sliding-window session renewal refreshes the Redis TTL on each authenticated request, with polling endpoints (such as the notification unread-count endpoint) excluded to avoid unnecessary Redis writes.

4.4.2 Email Verification and Rate Limiting

Registration and code-based login both require email verification. The platform uses the Resend HTTP API to deliver verification codes, with several hardening measures:

Code delivery. A 6-digit random verification code is generated and sent via a styled HTML email. The code is rendered in monospace font to reduce misreading errors between visually similar characters such as “0” and “O”. The code is stored in Redis with a 5-minute TTL and is deleted immediately after successful verification, enforcing single-use semantics.

Brute-force protection. After 5 consecutive incorrect attempts, the code is forcibly invalidated and the user must request a new one.

Three-tier rate limiting. A Redis-backed strategy prevents abuse at three levels, summarized in Table 8.

Table 8: Three-Tier Email Rate Limiting Strategy

Tier	Scope	Limit	Purpose
Tier 1	Per email address	1 request per 60 seconds	Prevent rapid-fire brute-force on a single target
Tier 2	Per IP address	5 requests per hour	Mitigate distributed enumeration attacks across multiple targets
Tier 3	Per email address	10 requests per day	Prevent persistent resource exhaustion and harassment

To ensure accurate IP attribution behind the Gateway and Nginx proxy, the User Service extracts the real client IP from the X-Forwarded-For and X-Real-IP headers rather than using the immediate TCP connection endpoint. The frontend disables the send button for 60 seconds after each request and displays a countdown.

4.4.3 RBAC and Department-Level Data Isolation

The platform implements a three-tier role-based access control model. Rather than hard-coding fine-grained permissions that would not generalize across institutions, a foundational role framework is provided, leaving detailed permission customization to the adopting bank.

Role definitions. Three roles govern all access:

- **ROLE_ADMIN.** Full system access: manage users, assign roles, configure departments, set clearance levels, adjust system settings including AI provider configuration and session timeout, and view all tasks platform-wide.
- **ROLE_DEPT_ADMIN.** Department-scoped access: manage membership and clearance levels within the administrator’s own department; create, edit, and delete department documents; review and approve document access escalation requests.
- **ROLE_USER.** Basic access: submit NL2SQL, RAG, and tool tasks via the Copilot or dedicated agent pages; query personal task history; view and request access to documents within the user’s department and clearance level.

Enforcement. At the controller level, Spring Security’s `@PreAuthorize` annotations restrict entire classes or individual methods to specific roles. The `AdminController` is accessible only to `ROLE_ADMIN`. The `DeptAdminController` accepts `ROLE_ADMIN` and

ROLE_DEPT_ADMIN, with programmatic checks further restricting department admins to their own department's data. At the Gateway level, the JWT filter extracts and forwards role claims; downstream services trust these headers because the Gateway is the sole entry point.

Document access model. Documents carry a department identifier and a security clearance level from 1 (Public) to 3 (Confidential). Access is governed by a multi-factor rule, shown in Table 9:

- **Global documents** (dept_id IS NULL): accessible to all authenticated users.
- **Department documents:** accessible only if the user belongs to the same department **and** the user's clearance level is greater than or equal to the document's security level.
- **Otherwise:** the document is marked as inaccessible and the user may submit a human-in-the-loop escalation request (RAG_APPLY notification), which routes to a department administrator for review. On approval, temporary access is granted with an audit trail.

Table 9: Document Access Control Rule — Pseudocode

```

1 FUNCTION listDocumentsForUser(userId, userDeptId, userClearance)
2   documents = queryAllDocuments()
3   result = []
4   FOR EACH doc IN documents:
5     IF doc.dept_id IS NULL THEN
6       doc.accessible = TRUE
7     ELSE IF doc.dept_id = userDeptId
8       AND doc.security_level <= userClearance THEN
9       doc.accessible = TRUE
10    ELSE
11      doc.accessible = FALSE
12      doc.escalationType = "RAG_APPLY"
13    END IF
14    result.add(doc)
15  END FOR
16  RETURN result
17 END FUNCTION

```

Notification subsystem. The notification system supports three message types. Chat notifications follow a simple Unread → Read flow. Document access escalation requests follow a Pending → Approved / Denied workflow, with the approving administrator's identity recorded for audit. SQL audit alerts carry a JSON payload with the suspicious

query and timestamp. The frontend polls for unread counts every ten seconds and displays them as badges.

4.4.4 Secure Communication

All external communication uses HTTPS with TLS 1.3. WebSocket connections are upgraded to WSS in production via Nginx configuration, with the backend checking the X-Forwarded-Proto header to redirect plain HTTP traffic. SQL injection is prevented through exclusive use of parameterized queries via PreparedStatement. User passwords are hashed with BCrypt at strength factor 10 and never stored in plaintext. API keys for external services (DeepSeek, Amap, Resend) are injected via environment variables and never exposed to the frontend.

4.5 Data Layer Architecture

For the data persistence layer, we adopted a multi-engine heterogeneous architecture. The main reason for not using a single database were the significant difference in storage requirements for different data types—storing vectors, relational data, and cache data all in MySQL would have created performance bottlenecks and architectural complexity. Instead, three specialized storage engines were deployed, each optimized for its specific data type.

All relational data resides on MySQL 8.0. We agreed on the database name `agent_platform`, character set `utf8mb4`, and collation `utf8mb4_unicode_ci`—while `general_ci` is more faster, it causes garbled characters in Chinese and emoji symbols. This design is essential for internationalization (including Traditional Chinese). The complete schema, including the base business tables and RAG metadata tables, is documented in Appendix A.

The user and permission management uses five tables: `sys_user` stores account information, passwords are hashed using BCrypt, and it's linked to `sys_department` as foreign key, also recording security levels (1-3). `sys_role` defines three role codes (`ROLE_ADMIN`, `ROLE_DEPT_ADMIN`, `ROLE_USER`). `sys_permission` defines six granular permissions covering user management, role management, and department management. `sys_user_role` is a many-to-many association table connecting users to roles, and `sys_role_permission` connects roles to granular permissions. The primary identity management process relies on the RBAC model described in the security section. For document management,

the `sys_document` table stores knowledge base content with clearance levels and department associations. Meeting room management uses `meeting_room` and `meeting_schedule` tables, where schedule records include `room_id` FK, `booker`, `start_time`, `end_time`, and attendee list.

During system initialization, we added a batch of seed data: four meeting rooms, five users with different roles and security levels, six departments, six permissions, a complete role allocation relationship tree, and eleven system documents covering banking compliance scenarios.

Cache and high-frequency operational data are handled by Redis 5.0+. Session tokens are stored in memory with configurable TTLs; atomic counting also rely on Redis when the gateway layer performs IP-level rate limiting (`INCR + EXPIRE` is the only strategy we use, it's simple, reliable, and effectively prevents race conditions). The JWT single-session mechanism is implemented through Redis keys `token:username: {username}`; on each login, the system invalidates any existing token for the same user.

Redis is also used for the tool's intelligent agent's schedule conflict detection. The key format is `tool:schedule:{userId}`. We store the meeting timestamps as scores using an ordered set. For overlapping queries, we use `ZRANGEBYSCORE` to retrieve entries whose `startEpoch` is within the query time window. The operation has logarithmic time complexity for both insertion and query. For MySQL-Redis consistency, we adopted a scheme of "write MySQL first, then write Redis, and delete the Redis key on failure," this is basically a simple cache-aside pattern, prioritizing strong consistency for persistent data.

Semantic retrieval for RAG agents requires vector support, so we separately introduced Milvus. The deployed container uses Milvus 2.6.15 in standalone mode. During the selection process, we considered several vector engines, and Milvus, with its relatively advanced four-tier architecture (access layer, coordinator, worker node, and object storage), ultimately stood out for enterprise deployment scenarios. Milvus's collection schema is dynamically configured at startup, with dimension matching the embedding model dimension. The default local BGE-M3 profile uses 1024-dimensional embeddings, COSINE similarity, and an HNSW index.

The RAG retrieval pipeline applies security filtering before generated answers are produced. The raw vector results returned by Milvus are not directly passed to the LLM;

instead, they are checked against the allowed document list computed from the user’s department, clearance level, role, and approved RAG_APPLY requests. Only permission-safe chunks are assembled into the final prompt.

Table 10 summarizes the data storage strategy.

Table 10: Multi-Modal Data Storage Strategy and Layered Access Patterns

Category	Engine	Primary Data Types	Access Pattern
Operational Layer	MySQL 8.0	Users, roles, departments, documents, meeting rooms, schedules, notifications	Transactional, ACID-compliant read/write
Caching Layer	Redis 5.0+	Session tokens, rate limit counters, metadata cache (hash), schedule conflicts (sorted sets)	High-concurrency, sub-millisecond reads
Object Storage	MinIO	PDF, DOCX, PPT files; Milvus backend objects	Document upload/download, RAG source document storage
Vector Layer	Milvus 2.6.15	Document chunk embeddings (BGE-M3, 1024 dimensions), source metadata	Semantic approximate nearest neighbor search

4.6 Technology Stack Rationale

After discussion, the technology stack of our system was determined based on four main principles: First, it should be able to be deployed locally to reduce reliance on external cloud services (because banking regulations require data to be stored on local servers); second, all components should work well with container technology (because we wanted the developer environment and the production environment to be as similar as possible); third, the technology stack should match the team’s current skill set (backend students are more familiar with Java than Python, so AI services had a separate Python module for LLM inference tasks); fourth, we preferred ecosystems that have long-term support and active communities (so we can find an answer on Stack Overflow when hitting a problem). Specifically, the system’s backend framework uses Java + Spring Boot. This choice was primarily based on its mature enterprise-level ecosystem. A strong typing system effectively reduces implicit errors caused by type mismatches; Spring Security provides fine-grained method-level permission control support for RBAC. Each microservice is con-

figured with an independent spring-boot-starter-parent, managed hierarchically through parent POMs to unify dependency versions.

Our gateway is Spring Cloud Gateway, rather than a traditional Servlet solution. (One issue with Servlets is that containers default to a blocking “one thread, one request” model. As the gateway acts as the hub for all service traffic, the extra thread overhead would have been substantial under high load. The reactive model is more suitable for the gateway’s role as a proxy and filter, given its high-concurrency scenario. The gateway performs identity verification through a custom JwtAuthGlobalFilter, repackaging the token payload (user ID, roles, and permissions) into the request headers, unifying the previously scattered authentication logic.

The AI inference service ultimately chose Python + FastAPI, deployed on port 8090. Python’s dominant position in the machine learning toolchain is unlikely to be replaced in the short term (compared to alternatives like Scala or R, its library ecosystem is dramatically richer). FastAPI offers performance close to Node.js while automatically generating Swagger documentation, which reduces the cost of inter-team interface coordination.

We settled on Vue 3 with TypeScript after weighing several options. While the team’s front-end skill levels were uneven, Vue 3’s learning curve proved less steep than alternatives, which mattered given our project schedule. TypeScript was not optional—it stopped numerous type errors at compile time that JavaScript would have silently let through. The Composition API gave us logic reuse that we actually used across all the pages that call agents that share the same skeleton of request → progress → result display logic, while the Options API would have forced code duplication.

Containerization via Docker and Docker Compose was our choice for environment parity. MySQL, Redis, and Milvus all run as containers, which eliminates the “it works on my machine” problem. The Java services are each packaged as thin JARs, with dependencies separated from the application code to reduce image rebuild time during iteration.

5 Core Agent Implementation

This chapter covers the three agent modules that form the platform’s intelligence layer—tool agent, code agent, and RAG agent. Each section discusses internal architecture, key algorithms, and data flow design. Pseudocode tables and sequence diagrams are included to illustrate the operational logic of each agent.

5.1 Tool Agent Implementation

This agent runs within a separate tool-agent service (Spring Boot, port 8083), providing three capabilities: meeting room management, schedule conflict detection, and route planning. Technically, business logic is divided into REST endpoints and specialized service classes; the AI intent module provides a unified entry point for natural language input.

5.1.1 Architecture and AI Intent Routing

The tool agent employs a dual-path design at the interaction layer: one side retains programmatic calls, while the other open a natural language entry point.

Programmatic calls use a separate REST endpoint, where the frontend or third-party system directly passes type parameters, making the logic straightforward. Natural language interactions are uniformly sent to the `/tool/execute` endpoint. The DeepSeek LLM (Starfire API, accessed via the iFlytek MaaS platform) identifies the user’s intent and extracts parameters from the natural language message. The request is then routed to the corresponding handler.

We separate the natural language entry point from the programmatic entry point, rather than using AI routing for everything, primarily for cost considerations. DeepSeek’s token consumption and response latency are much higher than a direct REST call (roughly 500–800ms versus 5–10ms for a local call). In cases where the frontend already knows exactly what operation is to be performed (button clicks), calling the AI again would be both unnecessarily slow and a waste of tokens—adding extra cost for no benefit.

We ultimately adopted the iFlytek MaaS (Model as a Service) platform, which provide an interface compatible with the OpenAI Chat Completions format. Therefore, the underlying HTTP client can directly use the standard OpenAI library call pattern without modifying the request body structure.

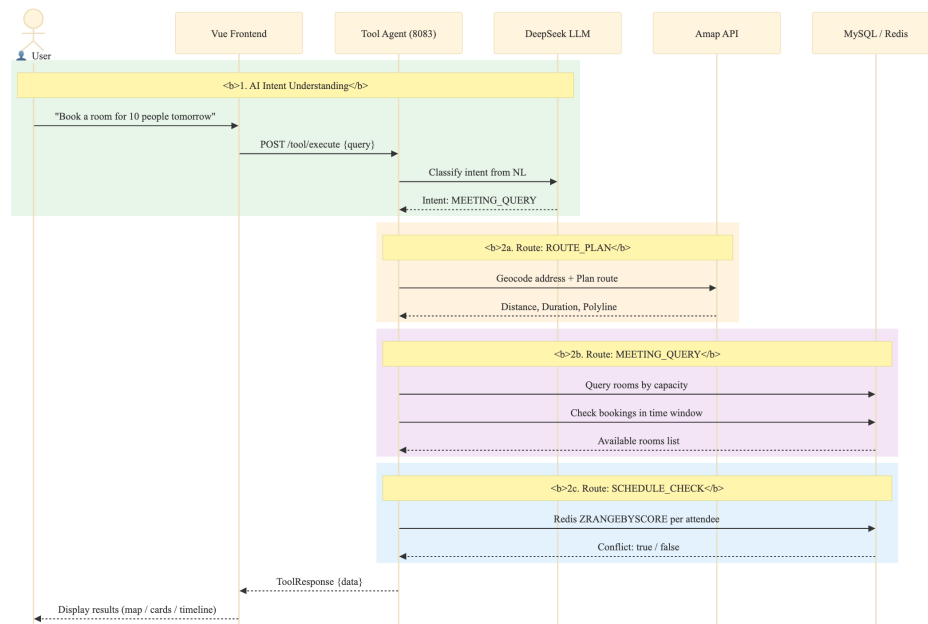


Figure 6: Tool Agent Task Sequence — DeepSeek AI Intent Routing to Specialized Handlers

Tables 11 and 12 show the AI intent routing pipeline where DeepSeek classifies user input into three intent types and dispatches to the corresponding handler, with a mock fallback on API failure.

Table 11: Pseudocode — handleAiIntent() AI Intent Classification and Routing

```
1 FUNCTION handleAiIntent(userInput: String): HandlerResult
2   systemPrompt = PromptBuilder.create()
3   .defineIntent("ROUTE_PLAN",
4     params = ["from", "to", "mode"],
5     description = "Route planning request")
6   .defineIntent("MEETING_QUERY",
7     params = ["date", "capacity", "equipment"],
8     description = "Meeting room search")
9   .defineIntent("SCHEDULE_CHECK",
10    params = ["timeRange", "attendees"],
11    description = "Schedule conflict detection")
12   .build()
13   response = deepSeekService.chat(
14     system = systemPrompt,
15     user = userInput,
16     format = "json_object",
17     temp = 0.3,
18     maxTok = 2048
19   )
20   intent = response.intentType
21   params = response.parameters
22   SWITCH intent:
23     CASE "ROUTE_PLAN":
24       RETURN routeHandler.handle(params.from, params.to, params.mode)
25     CASE "MEETING_QUERY":
26       RETURN meetingHandler.handle(params.date,
27         params.capacity, params.equipment)
28     CASE "SCHEDULE_CHECK":
29       RETURN scheduleHandler.handle(params.timeRange,
30         params.attendees)
31     DEFAULT:
32       THROW UnknownIntentException(intent)
33   END SWITCH
34 END FUNCTION
```

Table 12: Pseudocode — DeepSeekService.chat() LLM API Call with Fallback

```

1  FUNCTION chat(system, user, format, temperature, maxTokens): DeepSeekResponse
2      requestBody = {
3          model:          config.deepseek.model,
4          messages: [
5              {role: "system", content: system},
6              {role: "user",  content: user}
7          ],
8          response_format: {type: format},
9          temperature:     temperature,
10         max_tokens:      maxTokens
11     }
12     TRY
13         httpResponse = httpClient.send(
14             method: POST,
15             url:      config.deepseek.baseUrl + "/chat/completions",
16             headers: {
17                 "Authorization": "Bearer " + config.deepseek.apiKey,
18                 "Content-Type":  "application/json"
19             },
20             body:      toJSON(requestBody),
21             timeout: 60s
22         )
23         rawJson = httpResponse.choices[0].message.content
24         parsed  = objectMapper.readValue(rawJson, DeepSeekResponse)
25         RETURN parsed // {intentType, parameters}
26     CATCH Exception e:
27         log.error("DeepSeek API call failed", e)
28         RETURN fallbackMock() // demo data, source = "mock"
29     END TRY
30 END FUNCTION

```

The DeepSeek API configuration in application.yml uses environment variables with development defaults: AI_DEEPSEEK_API_KEY, AI_DEEPSEEK_BASE_URL (defaulting to https://maas-api.cn-huabei-1.xf-yun.com/v1), AI_DEEPSEEK_MODEL (defaulting to deepseek-v3), AI_DEEPSEEK_TIMEOUT (180s), and AI_DEEPSEEK_MAX_TOKENS (2000).

5.1.2 Meeting Room Management

For meeting room management, due to varying user permissions, we designed two sets of interfaces base on user permissions and functions:

The first set is for regular users. Regular users has a simple request: “Check if there are any available rooms for this time slot.” Therefore, our ToolController only includes a single GET request: /tool/meeting-rooms. Parameters include startTime, endTime, and capacity. The backend performs a three-phase availability check, as shown in the pseudocode in Table 13.

The second set, the management side, uses a completely different logic (although the sys-

tem administrator and department administrators maintain consistency). Because the Admin class needs to maintain meeting room resources, we provide a complete set of CRUD interfaces through MeetingRoomAdminController. In terms of security, it verifies the X-User-Roles header—this is simple but effective, because the header is injected by the gateway layer, not the client, and therefore cannot be forged.

Table 13: Pseudocode — queryRoomsWithStatus() Three-Phase Room Availability Algorithm

```

1 FUNCTION queryRoomsWithStatus(startTime, endTime, minCapacity): List<Map>
2   // === Phase 1: Query all active meeting rooms ===
3   query = new LambdaQueryWrapper<MeetingRoom>()
4     .eq(MeetingRoom::status, 1)
5   IF minCapacity <> NULL THEN
6     query.ge(MeetingRoom::capacity, minCapacity)
7   END IF
8   rooms = meetingRoomMapper.selectList(query)
9
10  // === Phase 2: Find booked room IDs in time range ===
11  overlapping = bookingMapper.selectList(
12    new LambdaQueryWrapper<MeetingSchedule>()
13      .lt(MeetingSchedule::startTime, endTime)
14      .gt(MeetingSchedule::endTime, startTime)
15  )
16  bookedRoomIds = {b.roomId | b IN overlapping}
17
18  // === Phase 3: Assemble availability response ===
19  result = []
20  FOR EACH room IN rooms:
21    available = room.id NOT IN bookedRoomIds
22    equipment = split(room.facilities, ",")
23    result.add({
24      id:          toString(room.id),
25      name:        room.roomName,
26      capacity:    room.capacity,
27      location:    room.building + ", Floor " + room.floor,
28      equipment:   equipment,
29      available:   available,
30      statusText:  IF available THEN "Available" ELSE "Booked",
31      nextBooking: findNextBooking(room.id, startTime)
32    })
33  END FOR
34  RETURN result
35 END FUNCTION

```

Table 14: Pseudocode — bookRoom() Atomic Conflict Check-and-Insert

```
1 FUNCTION bookRoom(roomId, startTime, endTime, booker, title): Long
2   // Step 1 -- Atomic conflict check
3   conflicts = scheduleMapper.selectCount(
4     new LambdaQueryWrapper<MeetingSchedule>()
5       .eq(MeetingSchedule::roomId, roomId)
6       .lt(MeetingSchedule::startTime, endTime)
7       .gt(MeetingSchedule::endTime, startTime)
8   )
9   IF conflicts > 0 THEN
10     RETURN FALSE // room already booked in this window
11   END IF
12   // Step 2 -- Insert new booking
13   schedule = new MeetingSchedule()
14   schedule.roomId = roomId
15   schedule.startTime = startTime
16   schedule.endTime = endTime
17   schedule.booker = booker
18   schedule.title = title
19   schedule.status = 1
20   scheduleMapper.insert(schedule)
21   RETURN schedule.id
22 END FUNCTION
23
24 FUNCTION getSchedulesForUsers(userIds, startTime, endTime): List<Schedule>
25   RETURN scheduleMapper.selectList(
26     new LambdaQueryWrapper<MeetingSchedule>()
27       .in(MeetingSchedule::booker, userIds)
28       .ge(MeetingSchedule::startTime, startTime)
29       .le(MeetingSchedule::endTime, endTime)
30   )
31 END FUNCTION
```

The MeetingRoom entity (@TableName(“meeting_room”)) maps to the database table with fields: id (Long, @TableId auto-increment), roomName (String), building (String, e.g., “Building A”), floor (String, e.g., “3F”), capacity (Integer), hasProjector/hasWhiteboard/hasVideoConf (Boolean), status (Integer, 0=available, 1=maintenance), and createTime (LocalDateTime).

Table 15 lists the Tool Agent REST API endpoints implemented in ToolController.

Table 15: Tool Agent REST API Endpoints

Method	Endpoint	Key Parameters	Description
POST	/tool/execute	toolType, naturalLanguage, parameters	Unified AI execution — DeepSeek LLM classifies intent and routes to handler
GET	/tool/meeting-rooms	startTime, endTime, capacity	Query rooms with time-based availability status and capacity filter
POST	/tool/meeting-room/book	roomId, booker, startTime, endTime, topic	Atomic check-then-insert room booking with conflict verification
POST	/tool/check-conflict	attendees[], startTime, endTime	Check schedule conflicts by attendee list via Redis ZRANGEBYSCORE
POST	/tool/schedule/create	userId, eventName, startTime, endTime	Create personal schedule (dual-write: MySQL room_id=0 + Redis sorted set)
POST	/tool/schedule/add	userId, eventId, startTime, endTime	Add schedule entry to Redis sorted set only
POST	/tool/schedule/remove	userId, eventId	Remove schedule entry from Redis sorted set by eventId prefix match
GET	/tool/schedules	date, users[]	Query schedules for specified users on a given date from MySQL
GET	/tool/my-schedules	X-User-Name header	Get authenticated user's bookings and personal schedules with room details
DELETE	/tool/my-schedule/{id}	X-User-Name header, path id	Cancel schedule: set status=0 in MySQL, remove from Redis if personal
GET	/tool/route	from, to, mode (default driving)	Plan route via Amap API with multi-level geocoding fallback and map data

5.1.3 Schedule Conflict Detection

Schedule conflict detection uses a dual-layer architecture: MySQL is the persistent store for all schedule data, while Redis serves as a fast query cache for temporal overlap detection. The ScheduleService handles CRUD operations on meeting_schedule records. Each schedule has a booker (the user who created it), optional room_id (0 for personal schedules that don't need a room), start_time/end_time, and an attendees field stored as a comma-separated list. Each user's schedules are indexed in Redis as a sorted set, where scores represent epoch timestamps and members encode event identifiers plus time ranges—this

structure allows $O(\log N)$ insertion and $O(\log N + M)$ range queries, where N is the number of schedules for that user and M is the number of results in the time window.

Figure 7 shows the schedule conflict detection algorithm.

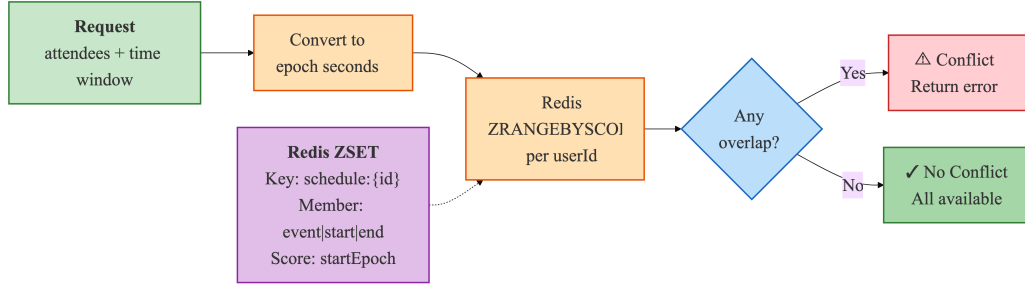


Figure 7: Schedule Conflict Detection — Redis Sorted Set-Based Temporal Overlap Algorithm

Each user’s schedules are stored in a Redis sorted set at key `tool:schedule:{userId}`. Schedule entries are encoded as members with the format “{eventId}|{startEpoch}|{endEpoch}”, where `startEpoch` and `endEpoch` are Unix timestamps. For conflict detection queries, `ZRANGEBYSCORE` retrieves all schedule entries whose `startEpoch` is within the specified query window. For each retrieved schedule, the algorithm checks if the stored `endEpoch` overlaps with the query range to filter out partial overlaps.

Tables 16, 17, and 18 show the schedule management subsystem including Redis sorted-set operations for temporal conflict detection, attendee-based conflict checking, and the MySQL-Redis dual-write consistency protocol.

Table 16: Pseudocode — Redis Sorted Set Schedule Operations (add / remove / checkConflicts)

```

1 FUNCTION addSchedule(userId, eventId, startTime, endTime)
2     key      = "tool:schedule:" + userId
3     startTs  = toEpochMillis(startTime)
4     endTs    = toEpochMillis(endTime)
5     member   = eventId + ":" + startTs + ":" + endTs
6     redisTemplate.opsForZSet().add(key, member, startTs)
7 END FUNCTION
8
9 FUNCTION removeSchedule(userId, eventId)
10    key      = "tool:schedule:" + userId
11    members = redisTemplate.opsForZSet().range(key, 0, -1)
12    FOR EACH member IN members:
13        IF member.startsWith(eventId + ":") THEN
14            redisTemplate.opsForZSet().remove(key, member)
15            BREAK
16        END IF
17    END FOR
18 END FUNCTION
19
20 FUNCTION checkConflicts(userId, startTime, endTime): List<String>
21    key      = "tool:schedule:" + userId
22    startTs  = toEpochMillis(startTime)
23    endTs    = toEpochMillis(endTime)
24    // Query schedules whose start time falls within window
25    overlapping = redisTemplate.opsForZSet()
26                .rangeByScore(key, startTs, endTs)
27    conflicts = []
28    FOR EACH member IN overlapping:
29        [evtId, memStartTs, memEndTs] = split(member, ":")
30        IF parseLong(memStartTs) < endTs THEN
31            conflicts.add(evtId)
32        END IF
33    END FOR
34    RETURN conflicts
35 END FUNCTION

```

Table 17: Pseudocode — POST /tool/check-conflict Attendee-Based Conflict Workflow

```

1 ENDPOINT POST /tool/check-conflict
2     Request: {attendees: String[], startTime: String, endTime: String}
3     Response: {hasConflict: Boolean, conflictUser: String?}
4
5 FUNCTION checkConflict(attendees, startTime, endTime): ConflictResult
6     FOR EACH userId IN attendees:
7         conflicts = scheduleService.checkConflicts(userId,
8             startTime, endTime)
9         IF conflicts IS NOT EMPTY THEN
10             // Early exit -- return first conflict found
11             RETURN {
12                 hasConflict: TRUE,
13                 conflictUser: userId,
14                 conflicts: conflicts
15             }
16         END IF
17     END FOR
18     RETURN {hasConflict: FALSE, conflictUser: NULL}
19 END FUNCTION

```

Table 18: Pseudocode — Schedule Dual-Write Pattern (MySQL + Redis Consistency)

```
1 // === Schedule Creation (dual-write) ===
2 ENDPOINT POST /tool/schedule/create
3 FUNCTION createSchedule(request): ScheduleResponse
4     // Write 1 -- Persist to MySQL
5     schedule = meetingRoomService.addPersonalSchedule({
6         roomId: 0, // personal schedule convention
7         booker: request.booker,
8         startTime: request.startTime,
9         endTime: request.endTime,
10        title: request.title,
11        status: 1
12    })
13    // Write 2 -- Cache to Redis for fast conflict queries
14    eventId = "event_" + schedule.id
15    scheduleService.addSchedule(request.booker, eventId,
16                                request.startTime, request.endTime)
17    RETURN {scheduleId: schedule.id, eventId: eventId}
18 END FUNCTION
19
20 // === Schedule Cancellation (dual-delete) ===
21 ENDPOINT POST /tool/schedule/cancel
22 FUNCTION cancelSchedule(scheduleId): void
23     // Step 1 -- Soft delete from MySQL (preserves audit trail)
24     scheduleMapper.update(null,
25        new LambdaUpdateWrapper<MeetingSchedule>()
26            .eq(MeetingSchedule::id, scheduleId)
27            .set(MeetingSchedule::status, 0)
28            .set(MeetingSchedule::updateTime, now())
29    )
30     // Step 2 -- Remove from Redis cache
31     scheduleService.removeSchedule(currentUser, "event_" + scheduleId)
32 END FUNCTION
```

5.1.4 Route Planning

The route planning feature integrates the Amap Open API through the AmapService class, which provides geocoding, route calculation, and map data for visualization. The GET /tool/route endpoint accepts origin, destination, and optional waypoint parameters, returning a route response containing path coordinates, turn-by-turn instructions, total distance, and estimated duration. geocoding (converting addresses to coordinates) is implemented through Amap's geocoding API, with detailed request parameters and LLM-based route description translation.

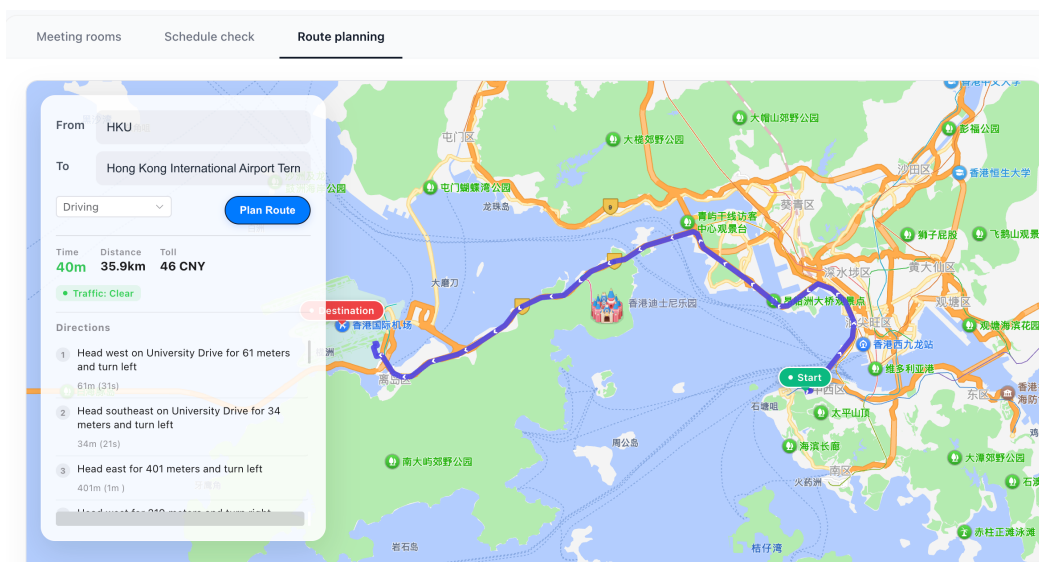


Figure 8: Route Planning Map Visualization — Interactive Map with Route Polyline and Navigation Steps

Table 19: Route Planning Three-Stage Geocoding and Navigation Pipeline

Stage	Method	API Endpoint	Input	Output	Fallback
1 – Geocoding	geocodeWith-Fallback()	GETrestapi.amap.com/v3/geocode/geo	Address string	Location (lng,lat)	POI text search, prepend “Beijing” + retry
2 – Route Calc	planDriving-Route()	GETrestapi.amap.com/v3/direction/driving	Origin + Dest coords, strategy (0–4)	Distance, duration, polyline steps	N/A — returns error to caller
3 – Enrichment	parseRoute-Result()	— (local processing)	Amap JSON response	Formatted text + coordinate arrays	N/A — keeps raw values on parse error

Table 20 shows the route geocoding pipeline with a three-level fallback strategy and LLM-based translation of navigation instructions.

Table 20: Pseudocode — geocodeWithFallback() Multi-Level Address Resolution

```
1 FUNCTION geocodeWithFallback(address: String): Location
2   TRY
3     response = httpGet(restapi.amap.com/v3/geocode/geo,
4       {key: apiKey, address: URL_ENCODE(address)})
5     IF response.geocodes IS NOT EMPTY THEN
6       RETURN parseLocation(response.geocodes[0].location)
7     END IF
8   CATCH: // fall through
9   TRY
10    // Fallback 1 -- POI text search
11    response = httpGet(restapi.amap.com/v3/place/text,
12      {key: apiKey, keywords: URL_ENCODE(address), offset: 1})
13    IF response.pois IS NOT EMPTY THEN
14      RETURN parseLocation(response.pois[0].location)
15    END IF
16  CATCH: // fall through
17  // Fallback 2 -- Prepend "Beijing" and retry geocoding
18  RETURN geocodeWithFallback("Beijing " + address)
19 END FUNCTION
20
21 FUNCTION translateRouteToEnglish(routeData): RouteData
22   fields = extractChineseFields(routeData)
23   TRY
24     translated = deepSeekService.chat(
25       system = "Translate to English, keep JSON structure",
26       user = toJSON(fields),
27       format = "json_object")
28     RETURN replaceUnits(translated)
29     // e.g., "km" instead of Chinese units
30   CATCH:
31     RETURN routeData // keep original Chinese
32   END TRY
33 END FUNCTION
```

On the frontend, the MapContainer.vue component loads the Amap JS API v2.0 SDK through script injection with the API key and security code. The component creates a map instance with zoom level 12, centered on the starting coordinate. An indigo polyline renders the route path. Green and red custom marker icons indicate the start and end points, respectively. Turning instructions are displayed in a scrollable list below the map, with each instruction showing the road name, distance, and a directional icon.

5.2 Code Agent Implementation

The Code Agent lets users generate executable SQL queries from natural language descriptions, connecting business users' domain expertise with the SQL knowledge needed for direct database interaction.

5.2.1 Architecture Overview

Table 21 shows the four-stage pipeline of Code Agent.

Table 21: Code Agent Four-Stage Natural Language to SQL Pipeline

Stage	Service Class	Key Operation	Security Mechanism
1 – Meta-data Ex-traction	MetadataCacheService	Query information_schema for table/column definitions; cache as JSON in Redis (key: code_agent:metadata:tables)	Provides whitelists (table names, column names) to the validation layer
2 – SQL Generation	OnnxCodeGeneration-Service	HTTP POST {question} → Python LLM inference server (port 8090); receives {sql, method}	N/A — generation only; output is untrusted at this stage
3 – SQL Validation	SqlValidationService	Five-layer defense-in-depth pipeline: (1) operation type, (2) forbidden keywords, (3) table whitelist, (4) column whitelist, (5) complexity limits	Blocks: non-SELECT, DDL/DML keywords, unrecognized tables/columns, overly complex queries
4 – SQL Execution	SqlExecutionService	Re-validate (defense in depth) → execute via JdbcTemplate with Prepared-Statement	Parameterized queries prevent injection; read-only connection enforced

The CodeAgentController exposes six endpoints: POST /code/query (one-shot generate and execute, returns FullResult with SQL, columns, rows, elapsedMs), POST /code/generate (preview only, generates SQL without execution), POST /code/execute (execute only, runs previously generated SQL), GET /code/metadata (returns cached schema metadata), POST /code/metadata/refresh (manually triggers schema refresh), and GET /code/health (returns service status and LLM connectivity).

Figure 9 shows the Code Agent use case diagram. Business analysts interact with the system by submitting natural language queries through the three-step frontend interface. The system processes requests by encoding database schema information, sending structured prompts to the Python LLM inference server, receiving generated SQL, running it through the five-layer validation pipeline, executing only validated queries via JdbcTemplate against MySQL, and returning results with the generated SQL and execution metadata.

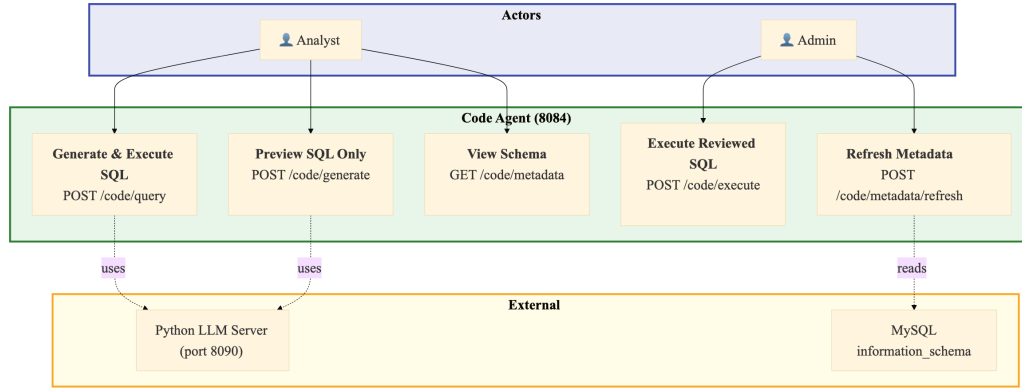


Figure 9: Code Agent Use Case Diagram — Natural Language to Safe SQL Execution

The use case covers schema-aware query generation, five-layer security validation, JdbcTemplate-based execution, and result presentation with export capabilities.

5.2.2 Python LLM Inference Server Integration

Table 22 shows the SQL generation HTTP proxy that forwards natural language questions to the Python LLM inference server and parses the returned SQL.

Table 22: Pseudocode — OnnxCodeGenerationService HTTP Proxy to Python Inference Server

```

1  // @Service, @ConditionalOnProperty(code-agent.onnx.enabled=true)
2  // Acts as HTTP proxy (NOT direct ONNX Runtime inference)
3  FUNCTION generateSQL(question: String): CodeGenerationResponse
4      payload = {question: question}
5      TRY
6          httpResponse = httpClient.send(
7              method: POST,
8              url:      config.serverUrl, // default: http://localhost:8090/infer
9              headers: {"Content-Type": "application/json"},
10             body:    toJSON(payload),
11             connectTimeout: 10s,
12             responseTimeout: 60s
13         )
14         responseBody = objectMapper.readTree(httpResponse.body)
15         sql = responseBody.get("sql").asText()
16         method = responseBody.get("method").asText()
17         IF sql IS BLANK THEN
18             RETURN failure("Generated SQL is empty")
19         END IF
20         RETURN success(sql, method)
21     CATCH Exception e:
22         log.error("SQL generation failed", e)
23         RETURN failure(e.message)
24     END TRY
25 END FUNCTION
26
27 // Design note: Any LLM serving framework can back port 8090
28 // as long as it accepts {question} and returns {sql, method}.
29 // Reference: Python inference server at data/infer_server.py
30 // hosts fine-tuned T5 model (data/fine-tuned-t5-banking/).
  
```


5.2.3 Five-Layer SQL Whitelist Validation

The SqlValidationService uses a defense-in-depth validation pipeline with five sequential layers, each of which can reject SQL that violates its security rule. The validation is configurable through CodeAgentProperties in application.yml.

Figure 10 shows the pipeline from natural language input to safe SQL execution. The five validation layers run in sequence.

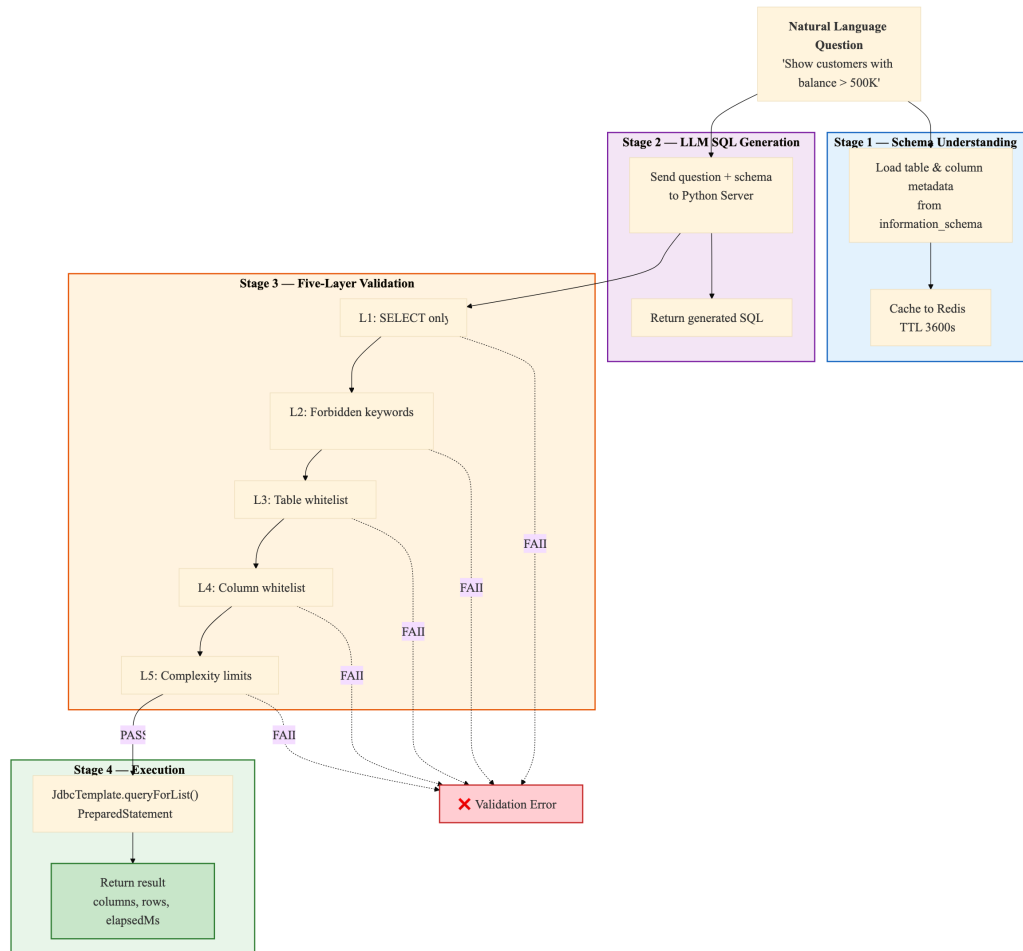


Figure 10: SQL Generation and Five-Layer Validation Pipeline

Tables 23, 24, 25, 26, and 27 show the five-layer SQL whitelist validation pipeline: operation type restriction, forbidden keyword blacklist, table name whitelist, column name whitelist, and query complexity limits.

Table 23: Pseudocode — validateOperation() SQL Operation Type Check (Layer 1)

```

1 FUNCTION validateOperation(sql: String): ValidationResult
2     normalized = collapseWhitespace(sql).trim().uppercase()
3     allowedOps = config.getAllowedOperations() // default: ["SELECT"]
4     FOR EACH op IN allowedOps:
5         IF normalized.startsWith(op) THEN
6             RETURN PASS
7         END IF
8     END FOR
9     RETURN FAIL("Only SELECT operations are permitted")
10 END FUNCTION

```

Table 24: Pseudocode — validateForbiddenKeywords() Keyword Blacklist Scan (Layer 2)

```

1 FUNCTION validateForbiddenKeywords(sql: String): ValidationResult
2     upperSQL = sql.uppercase()
3     // Default blacklist: DROP, DELETE, INSERT, UPDATE, ALTER,
4     // TRUNCATE, CREATE, EXEC, EXECUTE, UNION, INTO,
5     // LOAD_FILE, OUTFILE, DUMPFILE
6     FOR EACH keyword IN config.getForbiddenKeywords():
7         pattern = "\\b" + keyword + "\\b" // word-boundary regex
8         IF pattern IS FOUND IN upperSQL THEN
9             RETURN FAIL("Forbidden keyword detected: " + keyword)
10        END IF
11    END FOR
12    RETURN PASS
13 END FUNCTION

```

Table 25: Pseudocode — validateTableNames() Table Whitelist Verification (Layer 3)

```

1 FUNCTION validateTableNames(sql: String): ValidationResult
2     allowedTables = metadataCacheService.getAllowedTableNames()
3     IF allowedTables IS EMPTY THEN
4         log.warn("Metadata cache empty - skipping table whitelist")
5         RETURN PASS // skip when Redis/metadata unavailable
6     END IF
7     // Extract table references from FROM and JOIN clauses
8     pattern = "\\b(?:FROM|JOIN)\\s+?(\\w+)" // case-insensitive
9     extracted = findAllMatches(pattern, sql)
10    FOR EACH tableName IN extracted:
11        IF tableName NOT IN allowedTables THEN
12            RETURN FAIL(
13                "Table not allowed: " + tableName
14                + ". Allowed: " + allowedTables)
15        END IF
16    END FOR
17    RETURN PASS
18 END FUNCTION

```

Table 26: Pseudocode — validateColumnNames() Column Whitelist Verification (Layer 4)

```

1 FUNCTION validateColumnNames(sql: String): ValidationResult
2     // Skip for SELECT * or SELECT COUNT(*) queries
3     IF sql CONTAINS "SELECT *" OR sql CONTAINS "SELECT COUNT(*)" THEN
4         RETURN PASS
5     END IF
6     allowedColumns = metadataCacheService.getAllowedColumns()
7     IF allowedColumns IS EMPTY THEN
8         log.warn("Metadata cache empty - skipping column whitelist")
9         RETURN PASS // skip when metadata unavailable
10    END IF
11    // Extract column spec between SELECT and FROM
12    match = regex("SELECT\\s+(.+?)\\s+FROM", sql, CASE_INSENSITIVE)
13    IF match NOT FOUND THEN
14        RETURN PASS // cannot parse, allow through
15    END IF
16    columnsPart = match.group(1)
17    FOR EACH colSpec IN split(columnsPart, ","):
18        colRef = extractColumnReference(colSpec)
19        // Support both table.column and bare column
20        IF colRef IS qualified (t.col) THEN
21            IF colRef NOT IN allowedColumns[colRef.table] THEN
22                RETURN FAIL("Column not allowed: " + colRef)
23            END IF
24        ELSE
25            found = ANY table IN allowedColumns
26                WHERE colRef IN allowedColumns[table]
27            IF NOT found THEN
28                RETURN FAIL("Column not allowed: " + colRef)
29            END IF
30        END IF
31    END FOR
32    RETURN PASS
33 END FUNCTION

```

Table 27: Pseudocode — validateComplexity() Query Complexity Limits (Layer 5)

```

1 FUNCTION validateComplexity(sql: String): ValidationResult
2     // Count table references
3     tableCount = countMatches(
4         "\\b(?:FROM|JOIN)\\s+`?(\\w+)", sql)
5     maxTables = config.getMaxTablesPerQuery() // default: 3
6     IF tableCount > maxTables THEN
7         RETURN FAIL(
8             "Too many tables: " + tableCount
9             + " (max: " + maxTables + ")")
10    END IF
11    // Count conditions (AND/OR + 1 for base condition)
12    conditionCount = 1
13        + countMatches("\\bAND\\b", sql)
14        + countMatches("\\bOR\\b", sql)
15    maxConditions = config.getMaxConditions() // default: 10
16    IF conditionCount > maxConditions THEN
17        RETURN FAIL(
18            "Too many conditions: " + conditionCount
19            + " (max: " + maxConditions + ")")
20    END IF
21    RETURN PASS
22 END FUNCTION

```

Table 28 summarizes the five validation layers with their default configurations.

Table 28: SQL Whitelist Validation Rules (Five-Layer Defense)

Layer	Check	Default Configuration	Action on Violation
1. Operation	SQL must start with allowed operation	[“SELECT”] only	Block execution, return error with permitted operations
2. Keywords	Word-boundary regex blacklist scan	DROP, DELETE, INSERT, UPDATE, ALTER, TRUNCATE, CREATE, EXEC, EXECUTE, UNION, INTO, LOAD_ FILE, OUTFILE, DUMPFILE	Block execution, identify forbidden keyword
3. Table Names	FROM/JOIN tables vs. information_schema	All BASE TABLEs in agent_ - platform database	Block execution, show allowed table set
4. Column Names	SELECT columns vs. table column definitions	Per referenced table from metadata cache; skip on empty cache	Block execution, identify invalid column
5. Complexity	Table count + condition count	Max 3 tables, max 10 conditions (AND/OR count + 1)	Block execution, show actual vs. max counts

5.2.4 Metadata Caching and Execution

Tables 29, 30, and 31 show the schema metadata cache loading from information_schema to Redis, the cache lifecycle management with scheduled refresh, and the defense-in-depth SQL execution flow.

Table 29: Pseudocode — refreshAllMetadata() Schema Metadata Cache Loading

```
1 FUNCTION refreshAllMetadata(): Map<String, TableMetadata>
2   conn = dataSource.getConnection()
3   database = config.getDatabaseName()
4   // Query 1 -- All BASE TABLE names
5   tables = conn.query(
6     "SELECT TABLE_NAME FROM information_schema.TABLES
7       WHERE TABLE_SCHEMA = ? AND TABLE_TYPE = 'BASE TABLE'",
8     database)
9   metadataMap = {}
10  FOR EACH tableName IN tables:
11    // Query 2 -- Column metadata
12    columns = conn.query(
13      "SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE,
14        COLUMN_COMMENT, COLUMN_KEY
15        FROM information_schema.COLUMNS
16        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?
17        ORDER BY ORDINAL_POSITION",
18      database, tableName)
19    // Query 3 -- Table comment
20    comment = conn.querySingle(
21      "SELECT TABLE_COMMENT FROM information_schema.TABLES
22        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?",
23      database, tableName)
24    metadataMap[tableName] = {
25      columns: columns,
26      comment: comment
27    }
28  END FOR
29  conn.close()
30  // Serialize and cache to Redis
31  json = objectMapper.writeValueAsString(metadataMap)
32  redisTemplate.opsForHash().put(
33    "code_agent:metadata", "tables", json)
34  redisTemplate.expire(
35    "code_agent:metadata", config.getCacheTtl()) // default: 3600s
36  RETURN metadataMap
37 END FUNCTION
```

Table 30: Pseudocode — MetadataCacheManager Cache Lifecycle Management

```
1 // @Component - manages metadata cache lifecycle
2
3 // === Startup - initial cache load ===
4 @EventListener(ApplicationReadyEvent)
5 FUNCTION onApplicationReady():
6     TRY
7         metadataCacheService.refreshAllMetadata()
8         log.info("Metadata cache loaded successfully")
9     CATCH Exception e:
10        log.warn("Database not ready. Manually refresh via:")
11        log.warn("  POST /api/code/metadata/refresh")
12    END TRY
13 END FUNCTION
14
15 // === Periodic refresh - every 30 minutes ===
16 @Scheduled(fixedRate = 1800000)
17 FUNCTION scheduledRefresh():
18     TRY
19        metadataCacheService.refreshAllMetadata()
20    CATCH Exception e:
21        log.error("Scheduled metadata refresh failed", e)
22    END TRY
23 END FUNCTION
24
25 // === Fallback - direct query when Redis unavailable ===
26 FUNCTION getAllTableMetadata(): Map<String, TableMetadata>
27     TRY
28        cached = redisTemplate.opsForHash()
29            .get("code_agent:metadata", "tables")
30        IF cached NOT NULL THEN
31            RETURN deserializeJSON(cached)
32        END IF
33    CATCH: // Redis unavailable - fall through
34        // Direct DB query as fallback
35        RETURN refreshAllMetadata()
36 END FUNCTION
```

Table 31: Pseudocode — SqlExecutionService Defense-in-Depth SQL Execution

```
1 FUNCTION execute(sql: String): QueryResult
2     // Defense in depth - re-validate even from trusted pipeline
3     validation = sqlValidationService.validate(sql)
4     IF validation IS NOT PASS THEN
5         RETURN failure(validation.errorMessage)
6     END IF
7     // Execute via PreparedStatement (JdbcTemplate handles this)
8     TRY
9         rows = jdbcTemplate.queryForList(sql)
10        RETURN success({
11            columns:    extractColumnNames(rows),
12            data:        rows,
13            rowCount:    rows.size(),
14            elapsedMs:    measureElapsed()
15        })
16    CATCH DataAccessException e:
17        RETURN failure("Query execution failed: " + e.message)
18    END TRY
19 END FUNCTION
20
21 FUNCTION generateAndExecute(question, generationService): FullResult
22     genResult = generationService.generateSQL(question)
23     IF genResult IS FAILURE THEN
24         RETURN FullResult.failure(genResult.error)
25     END IF
26     execResult = execute(genResult.sql)
27     RETURN FullResult({
28         success:        execResult.success,
29         sql:            genResult.sql,
30         inferenceMethod: genResult.method,
31         columns:        execResult.columns,
32         rows:           execResult.data,
33         rowCount:        execResult.rowCount,
34         elapsedMs:       execResult.elapsedMs
35     })
36 END FUNCTION
```

5.3 RAG Agent Implementation

5.3.1 Architecture Overview

The RAG Agent is implemented as an independent Spring Boot microservice on port 8085. It is not a thin wrapper around a vector database; it owns the complete enterprise document question answering pipeline: document ingestion, chunking, embedding, vector indexing, permission-safe retrieval, grounded answer generation, citation construction, and query audit logging. The service is exposed through Spring Cloud Gateway under `/api/rag/**`. Gateway authentication is still the first security boundary: `JwtAuthGlobalFilter` validates the JWT token and forwards `X-User-Id`, `X-User-Name`, `X-User-Dept-Id`, `X-User-Roles`, and `X-User-Security-Level` headers to `rag-agent`.

Table 32 summarizes the internal pipeline. Compared with the Code Agent, whose main

risk is unsafe SQL execution, the RAG Agent’s main risk is unauthorized knowledge leakage. Therefore, permission calculation and audit logging are first-class stages rather than auxiliary checks.

Table 32: RAG Agent End-to-End Pipeline

Stage	Service / Component	Key Operation	Persisted State / Security Result
1 – Request Admission	Gateway + RagController	Authenticate JWT; forward user identity, department, role, and clearance headers	Reject unauthenticated requests before they reach retrieval logic
2 – Permission Snapshot	RagPermissionService	Compute accessible document IDs from department scope, clearance level, admin role, and approved RAG_APPLY notifications	Produces the allowed document set used for post-retrieval filtering
3 – Query Embedding	EmbeddingClient	Encode the user question with the active embedding profile	Records embedding profile, model, dimension, and collection for audit
4 – Vector Retrieval	VectorStoreService / REST Milvus adapter	ANN search with COSINE similarity and a 4x over-fetch candidate size	Returns candidate chunks that are not trusted yet
5 – Secure Filtering	RagQueryService	Remove chunks whose document IDs are outside the permission snapshot	Records retrieved document IDs and blocked document IDs
6 – Grounded Generation	Query prompt logic + RagLlmClient	Build evidence-only prompt; call OpenAI-compatible chat-completions endpoint	Produces answer text and citation references from allowed chunks only
7 – Response Audit	Query service + query log mapper	Return traceId, answer, citations, scores, and latency; persist query log	Enables later review of evidence, blocked docs, profile, and timing

Figure 11 shows the two parallel flows inside the RAG Agent: the indexing flow that prepares document vectors, and the query flow that retrieves evidence and generates answers. MySQL remains the source of record for document metadata and RAG audit data, while Milvus stores only vector-search-oriented chunk records.

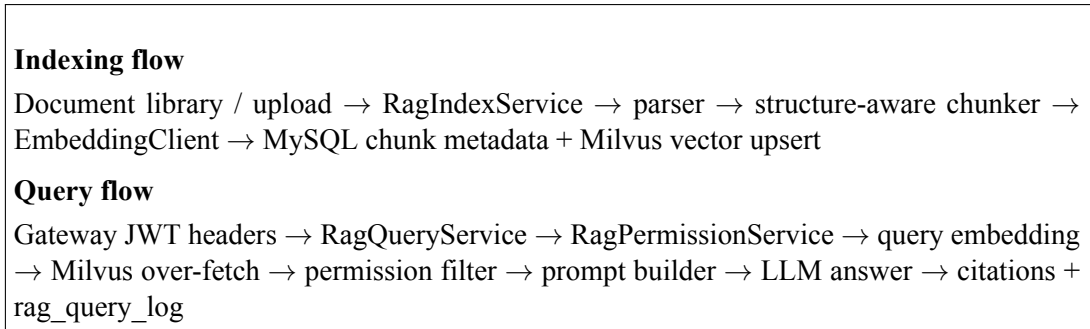


Figure 11: RAG Agent Architecture — Indexing Flow and Permission-Safe Query Flow

5.3.2 Data Structure and Storage Ownership

The existing sys_document table remains the document master table and stores title, content, dept_id, security_level, owner information, and document status. RAG-specific data is deliberately separated into metadata tables so that document management can evolve independently from vector retrieval.

Table 33: RAG Data Ownership Across MySQL and Milvus

Storage Object	Main Fields	Design Purpose
sys_document	title, content, dept_id, security_level, status	Source of truth for business document access control
rag_knowledge_base	name, description, owner, status	Groups documents into manageable knowledge bases
rag_source_document	source file path, parse status, index status, error message	Tracks uploaded files and parsing/index lifecycle
rag_document_chunk	document ID, chunk index, text, content hash, vector ID, embedding profile, model, collection, index status	Connects MySQL metadata to Milvus vector records
rag_query_log	user ID, question, answer, retrieved docs, blocked docs, latency, embedding profile	Audit trail for permission review and quality diagnosis
rag_index_task	task type, target document, status, progress, error	Operational tracking for rebuild, single-document index, and deletion jobs
Milvus collection	vector_id, document_id, chunk_id, chunk_index, dept_id, security_level, content_hash, chunk_text, embedding	ANN search over document chunks using the active embedding profile

Milvus is deployed as a standalone vector database with etcd for metadata coordination

and MinIO for object storage. The default local BGE-M3 profile uses 1024-dimensional embeddings, COSINE similarity, and an HNSW index. The collection name and vector dimension are not hard-coded in business logic; they are derived from the active embedding profile, allowing the local BGE-M3 profile and Qwen embedding profile to use independent vector collections without changing the query service.

5.3.3 Document Indexing and Chunking Pipeline

The indexing pipeline converts business documents into searchable semantic units. The important design decision is that chunk metadata is persisted before it is trusted by query-time generation. Each vector can be traced back to a document, a chunk row, an embedding profile, and a content hash.

Documents are processed with a structure-aware chunking strategy. The parser first extracts text from PDF, DOCX, PPT, or plain document content. The chunker then splits text by headings and paragraph boundaries to preserve hierarchy. When a segment exceeds the target size, it is divided into overlapping windows. The default configuration uses 700 tokens with a 100-token overlap. Each chunk inherits the source document's document ID, department ID, and security level.

Table 34 shows the indexing algorithm. The algorithm deletes old chunk vectors before inserting new records for a document, making document update behavior deterministic.

Table 34: Pseudocode — indexDocument() RAG Document Parsing, Chunking, and Vector Upsert

```

1  FUNCTION indexDocument(documentId: Long): IndexResult
2      task = indexTaskRepository.create("INDEX_DOCUMENT", documentId)
3      TRY
4          doc = documentRepository.findById(documentId)
5          IF doc IS unavailable THEN
6              RETURN failure("Document unavailable")
7          END IF
8
9          // Clean rebuild: remove old chunk rows and vectors
10         vectorStoreService.deleteByVectorIds(
11             chunkRepository.findVectorIds(documentId))
12         chunkRepository.deleteByDocumentId(documentId)
13
14         rawText = documentParser.parse(doc.filePath, doc.content)
15         sections = structureSplitter.split(rawText)
16         chunks = slidingWindowChunker.chunk(sections, 700, 100)
17
18         profile = embeddingProfileService.getActiveProfile()
19         vectors = embeddingClient.embedBatch(chunks.text, profile)
20
21         FOR i FROM 0 TO chunks.size - 1:
22             contentHash = sha256(chunks[i].text)
23             chunkRow = chunkRepository.insert(
24                 doc.id, i, chunks[i].text, contentHash, profile)
25             vectorId = buildVectorId(doc.id, chunkRow.id, profile.
26                 name)
27             vectorStoreService.upsert(
28                 collection = profile.collection,
29                 vectorId = vectorId,
30                 embedding = vectors[i],
31                 metadata = {
32                     documentId, chunkRow.id, i,
33                     doc.deptId, doc.securityLevel,
34                     contentHash, chunks[i].text
35                 })
36             chunkRepository.markIndexed(chunkRow.id, vectorId)
37         END FOR
38
39         indexTaskRepository.markSuccess(task.id, chunks.size)
40         RETURN success(chunks.size)
41     CATCH Exception e:
42         indexTaskRepository.markFailed(task.id, e.message)
43         RETURN failure(e.message)
44     END TRY
45 END FUNCTION

```

5.3.4 Permission-Safe Retrieval and Answer Generation

The query pipeline deliberately separates semantic relevance from authorization. Milvus is asked for a broader candidate set than the final top-K because some high-similarity

chunks may be inaccessible to the current user. The platform then filters candidates using the same document access model used by the normal document library.

Tables 35 and 36 show the complete query workflow. No retrieved chunk is allowed to enter the LLM prompt unless it passes the permission filter. This is the key difference between a normal RAG demo and an enterprise RAG system.

Table 35: Pseudocode — retrieveAllowedChunks() Permission-Safe Vector Retrieval

```

1 FUNCTION retrieveAllowedChunks(request, user): RetrievalResult
2     allowedDocIds = permissionService.listAccessibleDocumentIds(
3         user.id, user.deptId, user.roles,
4         user.securityLevel, includeApprovedRagApply = true)
5     IF allowedDocIds IS EMPTY THEN
6         RETURN RetrievalResult.empty("No accessible documents")
7     END IF
8
9     profile = embeddingProfileService.getActiveProfile()
10    queryVector = embeddingClient.embed(request.question, profile)
11    requestedTopK = clamp(request.topK, default = 5, max = 20)
12    candidateLimit = max(requestedTopK * 4, requestedTopK)
13    candidates = vectorStoreService.search(
14        profile.collection, queryVector, "COSINE",
15        limit = candidateLimit)
16
17    // 3. Enforce document permission after retrieval
18    allowedChunks = []
19    blockedDocIds = new Set()
20    FOR EACH candidate IN candidates:
21        IF candidate.documentId IN allowedDocIds THEN
22            allowedChunks.add(candidate)
23        ELSE
24            blockedDocIds.add(candidate.documentId)
25        END IF
26        IF allowedChunks.size == requestedTopK THEN
27            BREAK
28        END IF
29    END FOR
30
31    IF allowedChunks IS EMPTY THEN
32        queryLogRepository.saveNoEvidenceLog(user, request,
33            blockedDocIds)
34        RETURN RetrievalResult.empty("No permission-safe evidence")
35    END IF
36
37    RETURN RetrievalResult(allowedChunks, blockedDocIds, profile)
38 END FUNCTION

```

Table 36: Pseudocode — generateGroundedAnswer() Citation and Audit Workflow

```
1 FUNCTION generateGroundedAnswer(request, user): RagAnswer
2     startTime = now()
3     retrieval = retrieveAllowedChunks(request, user)
4     IF retrieval IS EMPTY THEN
5         RETURN noAnswer(retrieval.reason)
6     END IF
7
8     prompt = promptBuilder.buildEvidencePrompt(
9         request.question, retrieval.allowedChunks,
10        rule = "Answer only from context; cite sources")
11
12    llmResult = llmClient.chatCompletion(prompt)
13    citations = citationBuilder.fromChunks(
14        retrieval.allowedChunks, llmResult)
15
16    queryLogRepository.save({
17        user: user,
18        question: request.question,
19        answer: llmResult.answer,
20        retrievedDocIds: retrieval.allowedChunks.map(c -> c.
21            documentId),
22        blockedDocIds: retrieval.blockedDocIds,
23        embeddingProfile: retrieval.profile.name,
24        vectorCollection: retrieval.profile.collection,
25        latencyMs: elapsedMillis(startTime)
26    })
27    RETURN RagAnswer(llmResult.answer, citations, retrieval.
28        allowedChunks)
29 END FUNCTION
```

The answer generation step uses an OpenAI-compatible chat-completions endpoint configured by RAG_LLM_API_KEY, RAG_LLM_BASE_URL, and RAG_LLM_MODEL. The prompt has three parts: the user question, a numbered context block containing permission-safe chunks, and answer rules. The answer rules require the model to answer only from the provided context, cite chunk identifiers, state that no evidence was found when the context is insufficient, and never infer content from blocked documents. The response returned to the frontend contains traceId, answer text, citations, retrieved chunks, retrieved document IDs, blocked document IDs, topK, status, and latency.

Failure handling is part of the query contract rather than an unhandled exception path. If the user has no accessible documents, the service returns an empty evidence response with a controlled status and message. If Milvus, the embedding endpoint, or the LLM endpoint fails, RagQueryServiceImpl catches the exception, preserves the traceId, writes the status and message into rag_query_log, and returns a frontend-renderable failure response. This

keeps the RAG workbench debuggable without exposing inaccessible document content.

5.3.5 API Surface and Orchestration Integration

The RAG Agent exposes user-facing query endpoints, knowledge-base management endpoints, index maintenance endpoints, permission endpoints, and embedding-profile endpoints. All routes are protected by Gateway-level JWT authentication.

Table 37: RAG Agent REST API Endpoints

Endpoint	Role	Behavior
POST /rag/query	User	Runs permission-safe retrieval and returns traceId, answer, citations, chunks, blocked document IDs, topK, status, and latency
GET /rag/health, /rag/health/embedding	User / Admin	Reports service status, active Milvus collection, embedding readiness, LLM configuration, chunk size, and overlap
GET /rag/kb, POST /rag/kb	Admin	Lists and creates knowledge bases for RAG-managed source documents
POST /rag/kb/{kbId}/documents/ upload	Admin / Document owner	Uploads source files with department and security-level metadata, then triggers indexing
POST /rag/index/rebuild	Admin	Rebuilds all source-document vectors as an auditable index task
POST /rag/index/document/{id}, /reprocess	Admin / Document owner	Indexes or reprocesses one document with clean delete-and-upsert semantics
DELETE /rag/index/document/{id}	Admin / Document owner	Deletes chunk metadata and corresponding vectors for one document
GET /rag/index/tasks, /documents/status, /document/{id}/chunks	Admin / Document owner	Shows index task history, per-document index status, and persisted chunk details
GET /rag/permissions/documents	User	Returns the permission snapshot and accessible document list for the current user
GET /rag/embedding/profiles, /active	Admin	Lists configured embedding profiles and the active model, dimension, collection, and index status
POST /rag/embedding/profiles/{id}/ test, /activate	Admin	Tests an embedding profile and activates it with rebuild-required status tracking
POST /rag/access-request	User	Creates a RAG_APPLY notification for documents that require approval

The capability is also integrated into the task-service. When a RAG-type task is sub-

mitted, the task-service calls POST /rag/query with topK=5 and pushes progress through WebSocket: accepted, permission snapshot built, retrieval finished, answer generated, and completed. The final RAG response is stored in task_record.output, allowing the Task Center and Copilot to reuse the same evidence-grounded answer flow.

5.3.6 Frontend RAG Workbench and Citation Rendering

The /app/rag page is designed as an operational workbench rather than a simple chatbot. It includes a question input area, topK selector, loading states, answer rendering, citation cards, document metadata badges, and permission-filter diagnostics. Each citation card displays document title, chunk snippet, similarity score, security level, and document link. When candidate documents are retrieved but blocked by permission rules, the page shows the blocked count without exposing blocked content. If the user needs a restricted document, the access-request action creates a RAG_APPLY notification. After an administrator or department administrator approves it, RagPermissionService includes the approved document in the user's permission snapshot, so later retrieval can use that document as evidence. Copilot intent routing can redirect RAG-like questions to /app/rag?query=..., so a user can begin from the Copilot entry point and continue in the specialized RAG workbench with the question pre-filled.

5.3.7 Synchronization, Deployment, and Testing

Document-RAG synchronization follows the same clean lifecycle as the indexing pseudocode: document creation triggers single-document indexing; document update deletes old vectors and re-indexes the changed document; document deletion removes chunk rows and Milvus vectors. Content hashes allow the service to detect unchanged chunks and reduce unnecessary embedding work in rebuild tasks.

Docker Compose includes rag-agent, Milvus Standalone, etcd, MinIO, and rag-worker. Important environment variables include RAG_SERVICE_URI, MILVUS_HOST, MILVUS_PORT, RAG_EMBEDDING_PROVIDER, RAG_EMBEDDING_MODEL, RAG_EMBEDDING_DIM, RAG_CHUNK_SIZE_TOKENS, RAG_CHUNK_OVERLAP_TOKENS, RAG_LLM_API_KEY, RAG_LLM_BASE_URL, RAG_LLM_MODEL, RAG_MILVUS_COLLECTION, RAG_VECTOR_METRIC, and RAG_VECTOR_INDEX. Service dependency ordering ensures that Milvus, MinIO, etcd, and the embedding worker are available before rag-agent starts accepting queries.

Testing focuses on both quality and leakage prevention: admin users can retrieve high-clearance documents; standard users are restricted to same-department and clearance-allowed documents; low-clearance users cannot see confidential chunks; approved RAG_APPLY requests expand access; document updates refresh the index; document deletion removes vectors; Milvus or LLM failures return controlled error states; and the frontend citation panel renders evidence without exposing blocked document content.

5.4 Copilot — A Unified Intelligent Dispatcher Above the Three Agents

The three agents described in Sections 5.1–5.3 each solve a specific domain problem, but from the user’s perspective, they still require knowing *which* agent to use before typing a question. A bank employee asking “Which meeting rooms are free this afternoon?” should not need to understand that this goes to the Tool Agent rather than the Code Agent. The Copilot solves this by sitting above all three agents as a single natural language entry point: the user says what they need, and the Copilot figures out where it should go.

5.4.1 Design Rationale

The Copilot is not a fourth agent with its own domain capability. It is a **dispatcher** — its only job is to understand the user’s intent, route the request to the right agent, and present the result back in a unified chat interface. This design follows a simple principle: the platform has exactly one text box the user needs to care about, and it works across every page.

Table 38 summarizes the Copilot’s position relative to the three agents.

	Three Domain Agents	Copilot
Role	Execute domain-specific tasks (SQL, retrieval, tools)	Classify intent and dispatch
Input	Already-routed request with known type	Raw natural language, any topic
Output	Domain result (SQL rows, document excerpts, room lists)	Dispatch decision + agent result in chat
Scope	One agent = one capability	Cross-agent, cross-page
User interaction	Page-specific UI (tables, maps, panels)	Floating chat widget, always accessible

Table 38: Copilot vs. Three Domain Agents — Role Comparison

5.4.2 Architecture Overview

The Copilot spans three layers of the platform. On the frontend, a Vue 3 floating widget (CopilotWidget.vue) renders inside the global MainLayout, making it accessible from every page. On the backend, a Python Flask intent classifier (port 8090, shared with the Code Agent’s inference server) performs the core dispatch logic. Between them, the Task

Service provides asynchronous execution and WebSocket-based progress streaming for long-running agent calls.

Figure 12 shows how a user query flows through the Copilot system.

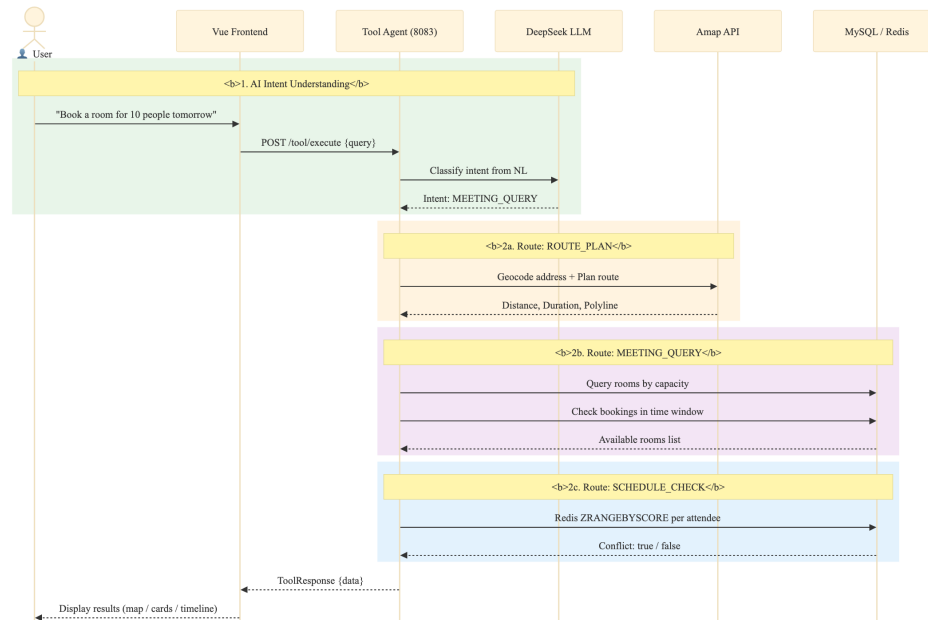


Figure 12: Copilot Query Flow — From Natural Language to Agent Execution

The flow has four stages. First, the user types a question into the Copilot widget. Second, the widget sends the question to `POST /api/dashboard/route`, which the Gateway rewrites to `/route` on the Python Flask server. Third, Flask runs a three-tier classification pipeline and returns an intent label. Fourth, the frontend dispatches based on the label: chat replies are shown inline, while domain tasks (TOOL, RAG) are submitted to the Task Service for asynchronous execution with progress streaming.

5.4.3 Three-Tier Intent Classification

The Python Flask `/route` endpoint is the Copilot’s brain. It implements a three-tier classification strategy that balances cost, latency, and accuracy, shown in Table 39.

Tier	Mechanism	What It Catches	Cost
Tier 1	Pattern blocklist (40+ regex rules)	Jailbreak attempts, non-banking topics, prompt injection	Zero (no API call)
Tier 2	Keyword greeting match	“Hello”, “What can you do”, simple greeting phrases	Zero
Tier 3a	LLM classification (temperature=0.1, max_tokens=10)	CODE, TOOL, RAG, CHAT, CLARIFY, SETTINGS	~10 tokens per query
Tier 3b	Keyword fallback (LLM unavailable)	Same six categories, lower accuracy	Zero

Table 39: Three-Tier Intent Classification Strategy

Tier 1 acts as a pre-screening safety net. Before any LLM call is made, the user’s input is checked against a hardcoded list of approximately 40 blocked patterns. These cover jailbreak prompts (e.g., “ignore previous instructions”), role-play attempts (“pretend you are a hacker”), code injection, and entirely off-topic requests (jokes, recipes, gaming). Blocked queries are caught instantly and receive a polite refusal without consuming any API tokens.

Tier 2 catches simple greetings. If the user says “Hello” or “What can you do?” or the Chinese equivalent, the system responds with a friendly introduction listing the four capabilities (database queries, policy Q&A, meeting and schedule tools, route planning) — again with zero LLM cost.

Tier 3a is the core classification layer. The user’s question is sent to the LLM with a system prompt defining six intent categories:

- **CODE:** Database queries, statistics, data lookups — anything that translates to SQL.
- **TOOL:** Meeting room booking, schedule conflict checking, route planning.
- **RAG:** Policy documents, regulations, concept definitions, compliance guidelines.
- **CLARIFY:** The user wants to book a room or plan a route, but has not provided enough parameters (e.g., no time specified for a meeting, no destination for a route).
- **SETTINGS:** Password changes, profile configuration.
- **CHAT:** Greetings, casual conversation, or anything outside the platform’s scope.

The LLM is configured with temperature 0.1 and max_tokens 10 — the response is a single word, not a sentence. This keeps classification deterministic and cheap. For CLARIFY and CHAT intents, a second LLM call generates a conversational reply with appropriate follow-up questions or explanations.

Tier 3b activates only when the LLM API is unreachable. A pure keyword-matching fallback scans for terms associated with each intent — for CODE: terms related to databases, statistics, and data queries; for TOOL: terms related to meeting rooms, bookings, and schedules — and defaults to RAG when no keywords match. This ensures the Copilot degrades gracefully rather than failing completely.

5.4.4 Frontend Dispatch and User Experience

After the intent is classified, the frontend widget takes over with type-specific handling, summarized in Table 40.

Intent	Widget Action	User Sees
CHAT	Display reply text directly in chat bubble	The LLM’s conversational response
CLARIFY	Save context, ask follow-up question	“Could you tell me what time you need the room?” — next user message prepends saved context
SETTINGS	Navigate to settings page	The settings form
CODE	Fire custom event, navigate to /app/code	The Code Agent page with the question pre-filled
TOOL	Submit task → WebSocket progress → result card	Animated processing bar with rotating status messages, then a result card (room list, conflict report, or route map)
RAG	Submit task → WebSocket progress → result card	A citation card with document titles, similarity scores, and security badges

Table 40: Intent Dispatch — Frontend Behavior by Classification Result

The Copilot widget itself is a floating component: a 52px circular icon button fixed to the bottom-right corner of every page. When clicked, it expands into a 440×620px chat panel with a morphing animation. This design means the user never leaves their current page to ask a question — the Copilot comes to them.

During agent execution, the input area is replaced by an animated processing bar. A puls-

ing blue dot, a water-ripple effect radiating from the input, and a set of status messages (“Analyzing your request...”, “Identifying intent...”, “Routing to the right agent...”, “Processing with AI model...”, “Gathering results...”, “Almost there...”) rotated every 1.5 seconds give the user continuous feedback. A live elapsed-time counter ticks every 100ms, so the user knows the system has not frozen.

When a task completes, the raw JSON output from the agent is transformed into a human-readable result card. Meeting queries show room names with colored availability dots, capacity badges, and location. Route plans show distance, duration, and traffic status. RAG answers show citation counts, blocked document counts, and a “View details” link. Conflict detections show either a green all-clear or an amber warning with the conflicting schedule details.

Session history is persisted in the browser’s localStorage under the key prefix `copilot_session_`, with a session index limited to 20 entries. Users can review past conversations or start a fresh session from the history panel.

5.4.5 Multi-Turn Clarification

The CLARIFY intent enables a simple but effective multi-turn dialogue pattern. If a user types “Book a meeting room,” the classifier recognizes that critical parameters are missing and returns CLARIFY. The widget saves the original query into a `sessionContextQuery` variable and displays the LLM’s follow-up question (e.g., “What time and how many people?”). When the user replies “Three o’clock, five people,” the widget prepends the saved context to form a combined query — “Book a meeting room [context: Three o’clock, five people]” — and re-classifies. This time, with all parameters present, the classifier returns TOOL and the request proceeds to the Tool Agent.

5.4.6 Cross-Page Communication

Because the Copilot widget is rendered once in `MainLayout` and persists across page navigation, it needs a mechanism to deliver agent results to whichever page the user is currently viewing. Two channels handle this:

1. **Browser Custom Events:** The widget dispatches `CustomEvent` objects on the window object — `copilot-tool-result` for Tool Agent results,

copilot-code-query for Code Agent queries. Target page components listen for these events and react accordingly.

2. **localStorage**: As a safety net for pages that mount *after* the event was dispatched (e.g., the widget navigated to a new page before the subscribing component existed), results are also written to localStorage keys. The target page checks these keys on mount and processes any pending result.

5.4.7 Second-Level Intent Classification in the Tool Agent

When the Copilot classifies a query as TOOL, the Task Service forwards it to the Tool Agent's /tool/execute endpoint with `toolType: "AI"`. At this point, a second classification happens inside the Tool Agent itself. The `handleAiIntent()` method sends the query to DeepSeek with a system prompt asking it to distinguish between three sub-intents: `MEETING_QUERY`, `SCHEDULE_CHECK`, and `ROUTE_PLAN`. The LLM returns a JSON object with the sub-intent and extracted parameters, which the Tool Agent uses to call the appropriate handler.

This two-level classification — Copilot first decides *which agent*, then the Tool Agent decides *which tool* — keeps each classifier's job simple. The Copilot's classifier only needs to distinguish broad domains; the Tool Agent's classifier only needs to distinguish between its own three tools. Neither classifier needs to understand the full complexity of the entire platform.

5.4.8 AI Provider Configuration Unification

Before the Copilot was introduced, each agent that called an LLM managed its own API key and endpoint configuration independently. The Tool Agent's DeepSeek service had its own `application.yml` block; the Python inference server read from a separate `ds_config.json` file; the RAG Agent had a third set of environment variables.

The Copilot unified all AI provider configuration under the user-service's `SystemConfigController`. A single admin UI (Settings page) accepts the provider name, base URL, model name, and API key. The key is AES-256 encrypted before storage in MySQL and cached in Redis under `sys:config:ai_provider`. Two REST endpoints serve the configuration:

- GET /user/config/ai-provider (public): Returns provider, base URL, and model — the API key is stripped for security. Used by the frontend to display the current model.
- GET /user/config/ai-provider/internal (internal, not exposed through the Gateway): Returns the full configuration including the decrypted API key. Called by the Python Flask classifier, the Tool Agent's DeepSeekService, and the RAG Agent. Because this endpoint is not routed through the Gateway, it is only reachable from within the Docker internal network.

With this unification, changing the LLM provider for the entire platform requires updating a single form — no configuration files to edit, no service restarts.

6 System Integration and Deployment

6.1 User Center and RBAC

The user center (user service, port 8081) integrates identity management, access control, department-level data isolation, as well as cross-cutting notification and document services. Table 41 shows the dual-mode login and JWT session management flow.

Table 41: Pseudocode — Dual-Mode Login and JWT Session Management

```
1 // === Dual-Mode Login ===
2 ENDPOINT POST /user/login -> {username, password, code?}
3 FUNCTION login(username, password, code): AuthResult
4     IF code IS PROVIDED THEN
5         // Mode 1 - Email verification code login
6         storedCode = redis.get("email:code:" + username)
7         IF storedCode IS NULL OR storedCode <> code THEN
8             THROW InvalidCodeException()
9         END IF
10        redis.delete("email:code:" + username) // single-use
11    ELSE
12        // Mode 2 - Password login with BCrypt verification
13        user = userMapper.selectByUsername(username)
14        IF user IS NULL OR NOT BCrypt.matches(password, user.password) THEN
15            THROW BadCredentialsException()
16        END IF
17    END IF
18    roles = roleMapper.selectByUserId(user.id)
19    perms = permissionMapper.selectByRoles(roles)
20    token = jwtService.generate({
21        sub: username,
22        roles: roles,
23        perms: perms,
24        deptId: user.deptId,
25        clearLv: user.clearanceLevel,
26        exp: now() + 24h
27    })
28    redis.set("session:active:" + username, token, {ttl: 30min})
29    RETURN {token, username, roles, permissions, realName}
30 END FUNCTION
31
32 // === Gateway JWT Validation (JwtAuthGlobalFilter, order = -100) ===
33 FUNCTION filter(exchange, chain):
34     IF requestPath IN whitelist THEN // /login, /register, /send-code
35         RETURN chain.filter(exchange)
36     END IF
37     token = extractBearerToken(request)
38     claims = jwtService.validateAndParse(token)
39     session = redis.get("session:active:" + claims.sub)
40     IF session IS NULL THEN THROW SessionExpiredException()
41     // Inject downstream headers
42     request.mutateHeader("X-User-Name", claims.sub)
43     request.mutateHeader("X-User-Roles", join(claims.roles))
44     request.mutateHeader("X-User-Permissions", join(claims.perms))
45     // Sliding-window renewal (exclude polling endpoints)
46     IF NOT requestPath.contains("/notification/unread-count") THEN
47         redis.expire("session:active:" + claims.sub, 30min)
48     END IF
49     RETURN chain.filter(exchange)
50 END FUNCTION
```

Access Control. Three roles (ROLE_ADMIN, ROLE_DEPT_ADMIN, ROLE_USER) and six granular permissions are used to control access to all protected endpoints. The six departments (Credit Department, Compliance Department, Asset Management Department, Retail Banking Department, Investment Banking Department, and Risk Control Department) provide organizational data scoping. A custom JwtAuthenticationFilter extracts permissions from JWT claims and registers them in Spring Security’s SecurityContextHolder. The @PreAuthorize annotation on each controller method can be declared with hasAuthority checks, while the abstraction level’s department admin controllers additionally verify that the requested resource’s dept_id matches the accessing user’s dept_id, enabling cross-cutting department-level data isolation.

6.2 Task Center and Orchestration

The Task Center (task-service, port 8082) is the platform’s asynchronous execution backbone. Its role is to accept work from any entry point — the Copilot widget, an agent page, or a future API integration — and ensure that work gets done reliably, with progress visible to the user throughout. Rather than letting each agent manage its own execution lifecycle, the Task Center centralizes queuing, retry, progress reporting, and result persistence.

6.2.1 Task Lifecycle and State Machine

Every piece of work in the platform is modeled as a task record with a four-state lifecycle, shown in Table 42.

State	Meaning	Transition Trigger
INIT	Record created, waiting for a worker thread	Task submitted via REST API or Web-Socket
RUNNING	Worker thread picked up the task, calling downstream agent	Executor dequeues the task
SUCCESS	Agent returned a valid result	Downstream agent responds with code 200
FAIL	Execution failed after all retries, or queue was full	Agent error, timeout, or RejectedExecutionException

Table 42: Task Lifecycle State Machine

The state machine is linear by design — no complex branching or rollback. If a task fails,

the retry mechanism re-executes from INIT state; it does not attempt to resume from the failure point. This keeps the implementation simple and predictable, which is appropriate for the current single-instance deployment.

6.2.2 Submission, Queuing, and Thread Pool

Tasks enter the system through two channels. The primary channel is `POST /task/submit`, called by the frontend after the Copilot has classified the user's intent. The Task Controller extracts the user's identity from the `X-User-Name` and `X-User-Roles` headers (set by the Gateway's JWT filter) and calls the Task Service. The secondary channel is through the WebSocket handler: when the Copilot widget connects to `/progress?taskId={id}`, it sends a JSON command with the task type and query, which triggers the same submission logic.

Table 43 shows the asynchronous task submission and queuing mechanism.

Table 43: Pseudocode — `submitTask()` Asynchronous Task Submission and Queuing

On submission, a `TaskRecord` is created in the `task_record` table with status `INIT` and `attemptCount` set to zero. The record is immediately persisted, so even if the service crashes before execution begins, there is an audit trail of what was requested. The task is then handed to a `ThreadPoolExecutor` configured as follows:

- **Core pool size:** 5 threads (always alive).
- **Maximum pool size:** 10 threads (created under load).
- **Work queue:** `LinkedBlockingQueue` with capacity 100.
- **Rejection policy:** `AbortPolicy` — throws `RejectedExecutionException` when the queue is full and all 10 threads are busy.

The choice of a bounded queue with `AbortPolicy` is deliberate. In a system without a message broker, an unbounded queue would mask overload by accepting tasks that cannot possibly complete in reasonable time. The 100-slot bound means the system explicitly

refuses work it cannot handle, and the frontend receives an immediate “System busy” response rather than an indefinite hang.

When a `RejectedExecutionException` is caught, the task’s status is set directly to `FAIL` with the message “System busy, please try again later” — the user sees this in the Copilot widget and can retry.

6.2.3 Agent Routing and Execution

Once a worker thread picks up a task, the `executeTaskOnce()` method routes it to the appropriate agent based on `taskType`, shown in Table 44.

taskType	Target Service	Endpoint	Request Body
CODE	code-agent (8084)	POST /code/query	{question: input}
TOOL / AI	tool-agent (8083)	POST /tool/execute	{toolType: "AI", naturalLanguage: input}
RAG	rag-agent (8085)	POST /rag/query	{question: input, topK: 5}

Table 44: Task Type to Agent Routing

All downstream calls use Spring’s `RestTemplate` with a 5-second connect timeout and 30-second read timeout. User identity headers (`X-User-Name`, `X-User-Roles`) are forwarded so that each agent can apply its own authorization logic. Service URIs are resolved from environment variables — `CODE_SERVICE_URI`, `TOOL_SERVICE_URI`, `RAG_SERVICE_URI` — with localhost fallbacks for development.

6.2.4 Retry with Exponential Backoff

Network calls to downstream agents can fail for transient reasons: a container restarting, a brief network hiccup, an LLM API rate limit. The retry mechanism in Table 45 handles these gracefully.

Table 45: Pseudocode — `executeWithRetry()` Exponential Backoff Loop

Each task gets up to 3 execution attempts. Between attempts, the worker sleeps for $1000 \times \text{attempt}$ milliseconds — the first retry waits 1 second, the second waits 2 seconds, the third waits 3 seconds. This spacing gives downstream services time to recover from transient failures without hammering them with immediate retries. If all 3 attempts fail, the task is marked FAIL and the error message is stored.

During retries, the WebSocket pushes progress updates (“Retrying... (2/3)”) so the user knows the system has not given up. The `attemptCount` field in the database is incremented on each attempt, providing an audit trail for debugging.

6.2.5 Progress Streaming and WebSocket Integration

A detailed discussion of the WebSocket implementation is provided in Section 6.3. From the Task Center’s perspective, the key integration point is that progress messages are pushed at domain-meaningful milestones rather than at fixed intervals. For CODE tasks: 10% (analyzing query), 30% (connecting to inference server), 50% (SQL generated, running validation), 80% (executing query), 100% (complete). For TOOL tasks: 10% (analyzing), 25% (intent routed to specific tool), 50% (AI processing), 80% (formatting result), 100%. For RAG tasks: 10% (checking permissions), 45% (vector retrieval), 70% (LLM answer generation), 90% (query logged), 100%.

These milestones are chosen to reflect what the user actually cares about — “Is it still thinking?” “Has it found the data?” “Is it almost done?” — rather than arbitrary percentage ticks.

6.2.6 Current Limitations

The Task Center’s design reflects a pragmatic trade-off for a single-instance system. Tasks are queued in memory via `LinkedBlockingQueue`; if the task-service restarts, queued-but-unexecuted tasks are lost (though their INIT records remain in MySQL as orphans). Worker threads block synchronously on downstream HTTP calls via `RestTemplate`, meaning concurrency is capped at the thread pool size of 10. WebSocket session storage is a local `ConcurrentHashMap`, which would not work across multiple instances. These are acceptable constraints for the current deployment scale, and replacing the in-memory queue with a message broker (RabbitMQ or Redis Streams) is a natural next step for production hardening.

Endpoint	Auth	Description
POST /task/submit	Headers	Submit an async task; returns task record with ID
GET /task/{id}	None	Poll task status and result by ID
GET /task/list	Headers	List tasks for the authenticated user (ordered by creation time descending)
GET /task/list/all	Admin only	List all tasks in the platform

Table 46: Task Service REST API Endpoints

RAG tasks use the same orchestration path. When the submitted task type is RAG, the task service forwards the question to rag-agent through POST /rag/query, stores the returned answer and citations in task_record.output, and pushes progress messages back to the frontend. This allows the Copilot, Task Center, and the dedicated RAG workbench to share the same backend capability instead of maintaining separate query implementations.

6.3 WebSocket Real-Time Communication

Long-running AI tasks need progress streaming beyond HTTP request-response. The platform uses WebSocket connections to push task progress from backend to frontend.

The WebSocket task progress architecture, illustrated in the Task Center section above, shows how long-running AI task progress is pushed from the backend TaskProgressWebSocketHandler to the frontend in real time via WebSocket connections proxied through the Spring Cloud Gateway.

Table 47 shows the WebSocket-based task progress streaming mechanism with step-by-step JSON push and frontend rendering.

Table 47: Pseudocode — TaskProgressWebSocketHandler Real-Time Progress Streaming

```
1 // === Backend: TaskProgressWebSocketHandler ===
2 // Registered at /tool/ws via WebSocketConfig
3
4 CLASS TaskProgressWebSocketHandler EXTENDS TextWebSocketHandler:
5     sessions = ConcurrentHashMap<String, WebSocketSession>()
6
7     FUNCTION afterConnectionEstablished(session):
8         taskId = extractQueryParam(session.uri, "taskId")
9         sessions.put(taskId, session)
10        executor.submit(() -> simulateProgress(taskId, session))
11    END FUNCTION
12
13    FUNCTION simulateProgress(taskId, session):
14        FOR step IN [1..5]:
15            sleep(500ms)
16            progress = {taskId: taskId, step: step, total: 5,
17                        message: "Processing step " + step + "/5"}
18            session.sendMessage(new TextMessage(toJSON(progress)))
19        END FOR
20        sessions.remove(taskId)
21    END FUNCTION
22 END CLASS
23
24 // === Frontend: WebSocket Client ===
25 // Singleton at utils/websocket.ts
26 FUNCTION connectWebSocket(taskId):
27     ws = new WebSocket(BASE_WS_URL + "/tool/ws?taskId=" + taskId)
28     ws.onmessage = (event) -> {
29         progress = JSON.parse(event.data)
30         agentThinking.updateSteps(progress)
31         IF progress.step = progress.total THEN
32             fetchTaskResult(taskId) // via AgentThinking.vue
33         END IF
34     }
35 END FUNCTION
36
37 // Gateway proxy: /ws/** -> tool-agent:8083/tool/ws
38 // JWT passed as query parameter; upgrade requests whitelisted
```

6.4 Containerized Deployment

Figure 13 shows the Docker Compose service topology, illustrating how each service (Gateway, four backend microservices, frontend, MySQL, Redis, and optionally Milvus with etcd and MinIO) runs in an independent container on an internal Docker network, with only the Gateway port exposed externally.

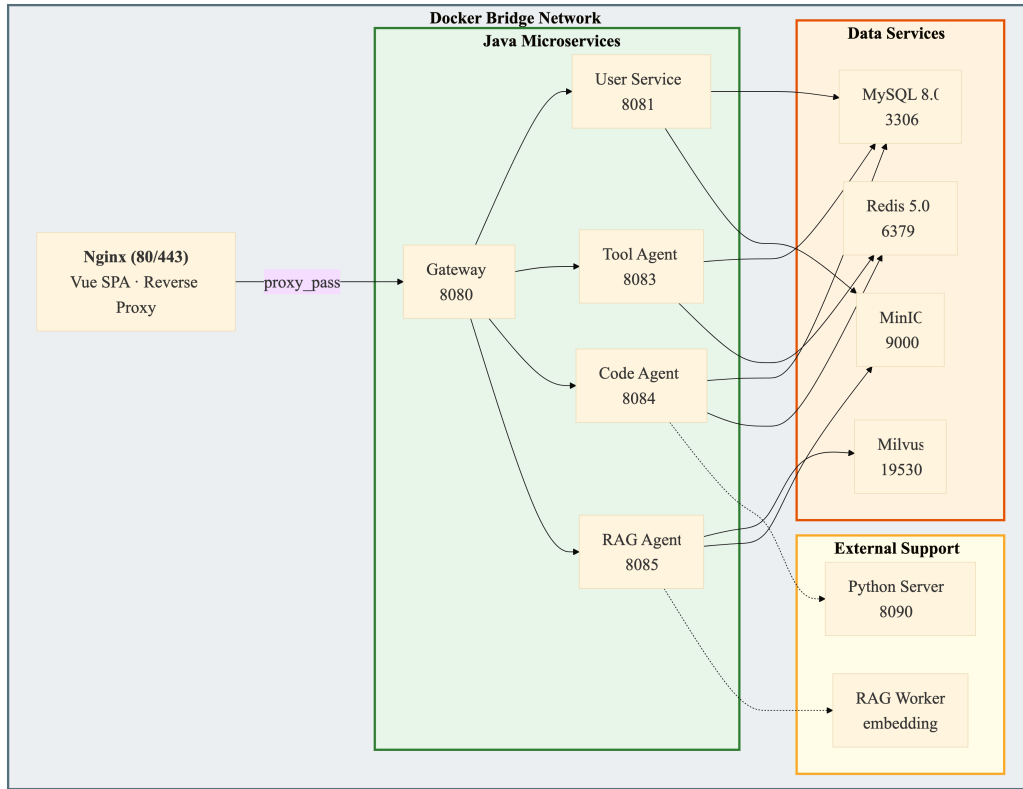


Figure 13: Docker Compose Service Topology

The platform is deployed through Docker Compose. Each service (Gateway, four backend services, frontend, MySQL, Redis, and optionally Milvus with etcd and MinIO) runs in an independent container on an internal bridge network. Only the Gateway (port 8080) is exposed. The frontend Nginx (port 80) serves static Vue 3 SPA files and proxies API and WebSocket requests to the Gateway. MySQL initialization is handled via a mounted SQL dump file. The Python inference server for the Code Agent runs in a separate container with FastAPI.

For RAG deployment, the Gateway maps `/api/rag/**` to `rag-agent:8085`. The `rag-agent` container depends on MySQL for metadata tables, Milvus for vector search, MinIO for document object storage, and `rag-worker` for local BGE-M3 embedding generation. Environment variables configure the active embedding provider, vector dimension, Milvus collection, chunk size, overlap, and LLM endpoint. This keeps the RAG subsystem deployable through the same Docker Compose command as the rest of the platform.

7 Discussion

7.1 Challenges and Resolutions

Frontend-Backend API Contract Alignment. During early integration, mismatches between the Vue 3 frontend’s expected API response structures and the Spring Boot backend’s actual response formats caused numerous runtime errors and slowed down development iteration speed. The resolution involved standardizing on a `Result<T>` generic wrapper (code, msg, data) for all microservice responses. Frontend Axios interceptors were configured to automatically unwrap the data field, so that component code only needs to handle domain data without manually parsing response layers. Additionally, Mock detect fallback is implemented on the backend so that frontend development could proceed independently without waiting for AI APIs to be fully available.

LLM API Integration for AI Intent Routing. The Tool Agent relies on the DeepSeek LLM for natural language intent classification, requiring prompt engineering to produce consistent, machine-parseable output. Several iterations of prompt design were needed. Early versions produced verbose natural language responses that couldn’t be reliably parsed, causing the intent routing switch statement to fail or select the wrong handler. A structured output format with explicit categories and parameter schemas was eventually adopted, combined with a mock-detect fallback that returns one of three hardcoded routes when the API is unreachable. Since the iFlytek MaaS platform is compatible with the OpenAI interface, the existing OpenAI library can be used directly.

Copilot Intent Classification. The Copilot adds a second layer of intent classification on top of the Tool Agent’s own routing. Getting the classifier to reliably distinguish between CODE, TOOL, and RAG intents required careful prompt design. The key insight was to keep the classification prompt minimal — six category definitions, temperature 0.1, max_tokens 10 — so the LLM returns a single word rather than a justification. Early attempts that asked the model to explain its reasoning produced verbose output that was both slower and harder to parse. The three-tier strategy (blocklist → keyword → LLM) also emerged from practical iteration: the blocklist and keyword tiers were added after observing that simple greetings and obvious jailbreak attempts were consuming LLM tokens unnecessarily. The CLARIFY intent was a later addition, added when testing showed that

users frequently submitted underspecified requests (“Book a room” without a time, “Plan a route” without a destination) and expected the system to ask for the missing information rather than fail silently.

Amap API Multi-Level Geocoding Fallback. When conducting the initial test of the informal address description, if there is a failure in geocoding, a multi-level rollback strategy needs to be adopted. For example, disambiguate by splitting regions, and use city-level approximate coordinates when detailed addresses cannot be found. After LLM translation, the Chinese navigation instructions are displayed.

Five-Layer SQL Validation Design. Designing a whitelist verification system requires striking a balance between security and availability. An overly strict validator will prevent legitimate analysis queries; an overly permissive one will expose the database to risk. After several rounds testing, the current five-layer architecture was found to be a good balance: operation type restriction (Layer 1) and forbidden keyword blacklist (Layer 2) provide broad security protection at the SQL system level; table and column whitelist (Layer 3 and 4) provide application-level data-scoping control; and query complexity limits (Layer 5) provide fine-grained resource governance and prevent performance-affecting Cartesian product joins. Each layer can be configured separately through the `application.yml` file, so different environments can adjust strictness according to their actual needs.

ONNX Model Optimization Path. Initially, it was planned to directly use ONNX Runtime for inference. However, this plan was eventually postponed and instead, a Python inference server based on HTTP was used. This happened for two reasons: first, ONNX Runtime’s support is not equally smooth across all model architectures, and the compatibility gap was significant; second, a separate Python server makes it easier to upgrade and replace models—different models can be swapped by changing the Python script only, without requiring changes to the Java service or the Spring context. The cost is additional network latency for the HTTP call, which will be addressed when local ONNX serving is revisited.

7.2 Comparison with Existing Solutions

Compared with cloud-based LLM API services, our platform achieves data sovereignty by processing locally and controlling the external API calls that involve the minimum

amount of data. Only user intent classification queries and geocoding requests go outside; all SQL generation, retrieval, and user data remains within the Docker network.

Compared with general multi-agent frameworks such as AutoGen and MetaGPT, our platform abandons the flexibility of dynamically allocating agent roles and temporary conversations. Instead, it adopts three long-running agents with fixed capabilities (code, RAG, and tool). This design trades theoretical flexibility for operational characteristics that are important in banking: deterministic behavior boundaries, access control at the agent level, and predictable resource usage.

Compared with independent RAG solutions, our integrated architecture enables unified access to three complementary AI capabilities (code generation, knowledge retrieval, and enterprise tool invocation) through a single platform. The trade-off is increased backend complexity; each agent is a separate microservice with its own dependencies, requiring coordinated deployment through Docker Compose and a Gateway for unified entry.

7.3 Project Limitations

Several limitations of the current implementation should be noted. The Code agent's Python HTTP server approach adds network latency and requires a separate Python process; local ONNX Runtime inference would reduce this overhead. The RAG agent is implemented end to end, but its retrieval precision depends on the embedding profile and the quality of indexed documents. The Tool Agent depends on external APIs (DeepSeek, Amap), and the unavailability of these services would degrade functionality to the mock fallback mode. The Task Center uses in-memory queuing via `LinkedBlockingQueue` rather than a message broker; while adequate for the current scale, this means queued tasks are lost on service restart. Single-instance deployment can handle approximately 50 concurrent users but not multi-node scaling; Kubernetes orchestration is planned as future work. RBAC is intentionally coarse-grained, so fine-grained custom permission design is left to the adopting institution. The Copilot's intent classifier is prompt-dependent — a significant change to the underlying LLM's behavior could require prompt adjustments, and the keyword fallback (Tier 3b) has lower accuracy than the LLM path.

8 Conclusion and Future Work

8.1 Summary of Achievements

This project presents a multi-agent AI platform for banking digital transformation. It addresses issues such as code generation accuracy, domain knowledge reliability, tool invocation robustness, and the data sovereignty demands unique to the banking industry. Through the collaborative work of three specialized agents, a cross-agent intelligent dispatcher, a five-layer defense architecture, and an on-premise Docker Compose deployment model, the platform provides banking personnel with unified AI capabilities accessible through a single natural language interface.

The Code agent implements a natural-language-to-SQL pipeline backed by a Python LLM inference server, combined with a five-layer SQL whitelist validation mechanism enforcing operation-type restrictions, keyword blacklisting, table/column whitelisting, and query complexity limits. The RAG agent combines Milvus vector retrieval, configurable embedding profiles, and an OpenAI-compatible LLM generation interface, adding document security-level filtering to ensure policy-compliant knowledge access with citations. The Tool agent implements a dual-path design (programmatic REST endpoints and AI intent routing through the DeepSeek LLM) with mock fallback support, providing meeting room management, Redis-based schedule conflict detection, and Amap-integrated route planning.

Above these three agents, the Copilot serves as a cross-page, multi-agent intelligent dispatcher. Its three-tier intent classification pipeline — a security blocklist, keyword greeting detection, and LLM-based six-category classification — routes natural language queries to the appropriate agent without requiring the user to understand which agent handles which domain. Multi-turn clarification enables the system to gather missing parameters through follow-up questions. A unified AI provider configuration center, accessible through a single admin UI, eliminates the configuration fragmentation that existed when each agent managed its own LLM settings independently.

The system integration layer provides JWT-based authentication with dual-mode login (password and email verification code), three-tier RBAC with department-level data isolation, HITL document access using a notification-based approval system, a Task Cen-

ter with asynchronous execution, exponential-backoff retry, and WebSocket real-time progress streaming, and containerized deployment via Docker Compose.

8.2 Future Work

Several directions for future work have been determined. Local ONNX Runtime inference for the Code Agent could reduce HTTP overhead. The Task Center should be upgraded from in-memory queuing to a message broker (RabbitMQ or Redis Streams) for production-grade durability. DevOps enhancements should include Prometheus metrics, Grafana dashboards, distributed tracing, Kubernetes orchestration for multi-node scaling, and automated CI/CD pipelines. The Copilot’s intent classifier could benefit from a feedback loop — tracking whether the dispatched agent actually handled the query successfully and using that signal to refine classification prompts. RBAC extensibility should allow adopting institutions to extend the three-tier role model with custom fine-grained permissions. The platform should be extended to support additional enterprise tools (email, HR systems, document management systems) through a pluggable tool registration mechanism, and to support additional database systems beyond MySQL.

References

- [1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium, 2018, pp. 3911–3921.
- [2] N. Pipitone and G. H. Alami, “LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain,” *arXiv preprint arXiv:2408.10343*, 2024.
- [3] N. Guha, J. Nyarko, D. E. Ho, C. Ré, A. Chilton, A. Narayana, A. Choksi, *et al.*, “LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models,” *arXiv preprint arXiv:2308.11462*, 2023.
- [4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [5] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Lin, Z. Liu, J. Liu, J. Zhang, Y. Lin, and W. Fan, “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework,” *arXiv preprint arXiv:2308.00352*, 2023.
- [6] A. Karras, L. Theodorakopoulos, C. Karras, A. Theodoropoulou, I. Kalliampakou, and G. Kalogeratos, “LLMs for Cybersecurity in the Big Data Era: A Comprehensive Review of Applications, Challenges, and Future Directions,” *arXiv preprint arXiv:2404.12345*, 2024.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for

Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *International Conference on Learning Representations (ICLR)*, 2023.

[11] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?” in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.

Appendix A: Database Schema

This appendix documents the complete database schema for the agent_platform database. All relational metadata tables use InnoDB engine, utf8mb4 character set with utf8mb4_unicode_ci collation. The schema covers user management, RBAC, document management, notification workflows, task tracking, tool resources, and RAG knowledge-base metadata.

Table 48: meeting_room — Meeting room resources

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique room identifier
room_name	VARCHAR(50), NOT NULL	Room display name (e.g., “Room 301”)
building	VARCHAR(100)	Building name (e.g., “Building A”)
floor	VARCHAR(20), NOT NULL	Floor identifier (e.g., “3F”)
capacity	INT, NOT NULL	Maximum occupancy
facilities	VARCHAR(200)	Equipment list (Projector, Whiteboard, etc.)
status	INT, DEFAULT 1	1=available, 0=maintenance
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	INT, DEFAULT 0	Logical delete flag

Table 49: meeting_schedule — Booking and schedule records

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique booking identifier
room_id	BIGINT, NOT NULL	FK to meeting_room.id (0 = personal schedule)
booker	VARCHAR(50), NOT NULL	Username of schedule owner
start_time	DATETIME, NOT NULL	Schedule start time
end_time	DATETIME, NOT NULL	Schedule end time
topic	VARCHAR(200)	Optional event description
status	INT, DEFAULT 1	1=active, 0=cancelled
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	INT, DEFAULT 0	Logical delete flag

Table 50: sys_user — User accounts

Column	Type	Description
id	BIGINT, PK, AUTO_- INCREMENT	Unique user identifier
username	VARCHAR(50), UNIQUE, NOT NULL	Login username (email format)
password	VARCHAR(100), NOT NULL	BCrypt-hashed password
real_name	VARCHAR(50)	Display name
role	VARCHAR(20), DE- FAULT 'user'	Legacy role description field
status	INT, DEFAULT 1	1=enabled, 0=disabled
dept_id	BIGINT	FK to sys_department
clearance_level	TINYINT, NOT NULL, DEFAULT 1	1=Public, 2=Internal, 3=Confidential
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete (0=normal, 1=deleted)

Seed accounts (all passwords: 123456): admin (ROLE_ADMIN, clearance L3); credit_mgr (ROLE_DEPT_ADMIN, L2, dept_id=1); credit_staff (ROLE_USER, L1, dept_id=1); compliance_mgr (ROLE_DEPT_ADMIN, L2, dept_id=2); compliance_staff (ROLE_USER, L1, dept_id=2).

Table 51: sys_role — System roles

Column	Type	Description
id	BIGINT, PK, AUTO_- INCREMENT	Unique role identifier
role_code	VARCHAR(50), UNIQUE, NOT NULL	Role code (ROLE_ADMIN, ROLE_DEPT_ADMIN, ROLE_USER)
role_name	VARCHAR(50), NOT NULL	Human-readable role name
description	VARCHAR(250)	Role description
status	INT, DEFAULT 1	1=enabled, 0=disabled
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete flag

Table 52: sys_permission — Granular permissions

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique permission identifier
perm_code	VARCHAR(100), UNIQUE, NOT NULL	Permission code (e.g., “user:view”)
perm_name	VARCHAR(100), NOT NULL	Human-readable permission name
resource_path	VARCHAR(200)	API endpoint path
method	VARCHAR(10), DEFAULT ‘*’	HTTP method (GET/POST/PUT/DELETE/*)
type	TINYINT, DEFAULT 1	1=menu, 2=API/button
parent_id	BIGINT, DEFAULT 0	Parent permission ID (hierarchical)
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete flag

Seed (6): user:view, user:role, user:status, role:view, task:submit, task:query.

Table 53: sys_user_role & sys_role_permission — Association tables

<i>sys_user_role</i>		
Column	Type	Description
user_id	BIGINT, NOT NULL	FK to sys_user.id
role_id	BIGINT, NOT NULL	FK to sys_role.id
PRIMARY KEY		(user_id, role_id)
<i>sys_role_permission</i>		
Column	Type	Description
role_id	BIGINT, NOT NULL	FK to sys_role.id
perm_id	BIGINT, NOT NULL	FK to sys_permission.id
PRIMARY KEY		(role_id, perm_id)

Table 54: sys_department — Organizational structure

Column	Type	Description
id	BIGINT, PK, AUTO_- INCREMENT	Unique department identifier
dept_name	VARCHAR(100), NOT NULL	Department name
description	VARCHAR(250)	Department description
parent_id	BIGINT, DEFAULT 0	Parent department ID (hierarchical)
status	INT, DEFAULT 1	1=enabled, 0=disabled
create_time	DATETIME	Record creation timestamp

Seed (6): Credit Department, Compliance Department, Asset Management Department, Retail Banking Department, Investment Banking Department, Risk Control Department.

Table 55: sys_document — Knowledge base documents

Column	Type	Description
id	BIGINT, PK, AUTO_- INCREMENT	Unique document identifier
title	VARCHAR(100), NOT NULL	Document title
content	TEXT, NOT NULL	Document body (Markdown)
dept_id	BIGINT	FK to sys_department (NULL = global)
security_level	TINYINT, NOT NULL, DEFAULT 1	1=Public, 2=Internal, 3=Confidential
file_type	VARCHAR(20), DEFAULT 'MARKDOWN'	MARKDOWN / PDF / DOCX / PPT
file_size	BIGINT	Original file size in bytes
minio_object_key	VARCHAR(500)	MinIO object key for uploaded files
parse_status	VARCHAR(20)	PENDING / DONE / FAILED
create_time	DATETIME	Record creation timestamp

Table 56: sys_notification — Multi-type notification system

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique notification identifier
sender_id	BIGINT, NOT NULL	FK to sys_user.id
receiver_id	BIGINT, NOT NULL	FK to sys_user.id
title	VARCHAR(150), NOT NULL	Notification title
content	TEXT, NOT NULL	Notification body
notify_type	VARCHAR(50), DEFAULT 'CHAT'	CHAT / RAG_APPLY / SQL_AUDIT / BUG_REPORT
status	INT, DEFAULT 1	1=Unread, 2=Read/Pending, 3=Approved, 4=Rejected
payload	TEXT	JSON payload for structured metadata
opinion	TEXT	Approval review comments
parent_id	BIGINT	Parent notification ID (threading)
thread_id	BIGINT	Root conversation thread ID
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete flag

Table 57: sys_config — System configuration

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique config entry identifier
param_key	VARCHAR(100), UNIQUE, NOT NULL	Configuration key
param_value	VARCHAR(500), NOT NULL	Configuration value
description	VARCHAR(250)	Human-readable description
update_time	DATETIME	Auto-updated on modification

Table 58: task_record — Agent task lifecycle tracking

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique task identifier (starts at 1000)
task_type	VARCHAR(20), NOT NULL	Task type: CODE / RAG / TOOL
status	VARCHAR(20), NOT NULL, DEFAULT 'INIT'	INIT / RUNNING / SUCCESS / FAIL
user_id	BIGINT, NOT NULL	FK to sys_user.id
input	TEXT, NOT NULL	User's natural language prompt
output	TEXT	AI agent output result
error_msg	TEXT	Error details on failure
attempt_count	INT, NOT NULL, DEFAULT 0	Retry count
elapsed_time	INT, DEFAULT 0	Execution time in milliseconds
created_at	DATETIME	Record creation timestamp
updated_at	DATETIME	Auto-updated on modification

Table 59: RAG knowledge base tables

<i>rag_knowledge_base</i>		
Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique KB identifier
name	VARCHAR(120), NOT NULL	Knowledge base display name
description	VARCHAR(500)	Knowledge base description
owner_username	VARCHAR(100)	Creator or owner username
dept_id	BIGINT	Department scope (NULL = platform-wide)
visibility	VARCHAR(30), NOT NULL, DEFAULT 'DEPARTMENT'	PRIVATE / DEPARTMENT / GLOBAL
security_level	TINYINT, NOT NULL, DEFAULT 1	Default security level for documents
status	VARCHAR(30), NOT NULL, DEFAULT 'ACTIVE'	ACTIVE / ARCHIVED / DELETED
document_count	INT, NOT NULL, DEFAULT 0	Number of documents in this KB
chunk_count	INT, NOT NULL, DEFAULT 0	Total chunk count
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification

<i>rag_source_document</i>		
Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique source document identifier
kb_id	BIGINT, NOT NULL	FK to rag_knowledge_base.id
sys_document_id	BIGINT	Linked sys_document.id
title	VARCHAR(200), NOT NULL	Display title
original_file_name	VARCHAR(255), NOT NULL	Uploaded file name
file_type	VARCHAR(32)	File extension (pdf, docx, pptx, txt)
mime_type	VARCHAR(120)	Browser-reported MIME type
file_size	BIGINT	File size in bytes
storage_provider	VARCHAR(30), NOT NULL, DEFAULT 'minio'	Storage backend
storage_bucket	VARCHAR(120)	MinIO bucket name
storage_object_key	VARCHAR(500)	MinIO object key
content_hash	VARCHAR(64)	SHA-256 hash of original file bytes
parsed_text	LONGTEXT	Parsed text cache for chunking
status	VARCHAR(30), NOT NULL, DEFAULT 'ACTIVE'	ACTIVE / DELETED
parser_status	VARCHAR(30), NOT NULL, DEFAULT 'UPLOADED'	UPLOADED / PARSED / PARSE_PENDING / PARSE_FAIL
index_status	VARCHAR(30), NOT NULL, DEFAULT 'PENDING'	PENDING / INDEXED / INDEX_FAIL

Appendix B: API Documentation (Selected Endpoints)

POST /api/user/login → {username, password, code?} → {token, username, roles[], permissions[], realName} (JWT signed, 24h expiry; code field optional for verification code login mode)

POST /api/user/send-code → {email} → Sends 6-digit code via Resend (5min TTL, 60s cooldown, daily cap 5)

POST /api/user/register → {username, password, code} → {token, username, roles[], permissions[]}

GET /api/user/me → {id, username, realName, roles[], permissions[], deptId, securityLevel}

PUT /api/user/profile → {realName} → Update display name

PUT /api/user/password → {oldPassword, newPassword} → Change password

User Service (Port 8081) — Authentication:

POST /api/user/login → Authenticate and receive JWT token

POST /api/user/register → Register new account

POST /api/user/send-code → Send email verification code

GET /api/user/me → Get current user profile

PUT /api/user/profile → Update display name

PUT /api/user/password → Change password

Admin (ROLE_ADMIN only):

GET /api/admin/users → [UserResponse[]]; POST /api/admin/user/role → {userId, roleId} → Clears old, assigns new

PUT /api/admin/user/status → {userId, status} → Enable/disable; PUT /api/admin/user/dept → {userId, deptId}

PUT /api/admin/user/security-level → {userId, securityLevel} → Set clearance (1-3)

GET /api/admin/roles → Role[]; GET /api/admin/permissions → Permission[]

Dept Admin (ROLE_DEPT_ADMIN/ROLE_ADMIN):

GET /api/user/dept-admin/members → (deptId? for ADMIN) → [UserResponse[]]

GET /api/user/dept-admin/candidates → Users with null dept_id

POST /api/user/dept-admin/add-members → {userIds[], deptId?} → Batch assign department

Documents:

GET /api/user/document/list → Clearance-filtered [DocumentResponse[]] (accessible boolean, content or “Access Restricted”)

POST /api/user/document/create → {title, content, deptId, securityLevel} → ADMIN/DEPT_ADMIN only

PUT /api/user/document/update → {id, title, content, securityLevel} → ADMIN/DEPT_ADMIN (own dept)

DELETE /api/user/document/delete/{id} → ADMIN only

GET /api/user/document/{id} → Document detail (with clearance check)

Notifications:

GET /api/user/notification/list → (status?, notifyType?) → [NotificationResponse[]] enriched with sender/receiver names

GET /api/user/notification/unread-count → Integer count (status=0); POST /api/user/notification/mark-read/{id}

POST /api/user/notification/approve/{id} → replyMessage? → Approve HITL request;

POST /api/user/notification/deny/{id} → replyMessage? → Deny

Tool Agent (Port 8083):

POST /api/tool/execute → {toolType, parameters?, naturalLanguage} → AI-routed result (DeepSeek intent classification)

GET /api/tool/meeting-rooms → (startTime?, endTime?, capacity?) → [Room[]] with availability status

POST /api/tool/book-room → {roomId, startTime, endTime, title, attendees} → Booking confirmation

POST /api/tool/check-conflict → {attendees[], startTime, endTime} → {hasConflict, conflictUser, message}

GET /api/tool/schedule/user/{userId} → (startDate?, endDate?) → [Schedule[]]; POST /api/tool/schedule → Create personal schedule

PUT /api/tool/schedule/{id} → Update; DELETE /api/tool/schedule/{id} → Delete

GET /api/tool/route → (origin, destination, waypoints?) → {coordinates[], steps[], distance, duration}

Admin (ROLE_ADMIN):

CRUD /api/tool/admin/meeting-rooms → X-User-Roles header checked; validations: roomName req, floor req, capacity>0

CRUD /api/tool/admin/schedules → X-User-Roles header checked; validations: booker req, startTime<endTime

Code Agent (Port 8084):

POST /api/code/query → {question, database?} → FullResult {success, sql, columns[], rows[], rowCount, elapsedMs, inferenceMethod}

POST /api/code/generate → {question} → {success, sql, inferenceMethod}

POST /api/code/execute → {sql} → {success, columns[], rows[], rowCount, elapsedMs}

GET /api/code/metadata → Cached schema {tables: {}}; POST /api/code/metadata/refresh → Force refresh; GET /api/code/health → Status

RAG Agent (Port 8085):

POST /api/rag/query → {question, topK?} → {traceId, answer, citations[], chunks[], retrievedDocumentIds[], blockedDocumentIds[], topK, latencyMs, status}

GET /api/rag/health → RAG service, Milvus, embedding, and LLM readiness summary

GET /api/rag/health/embedding → Active embedding profile readiness and dimension check

GET /api/rag/permissions/documents → Current user's accessible document snapshot

POST /api/rag/index/rebuild → Trigger full RAG index rebuild

POST /api/rag/index/document/{documentId} → Index one document; POST /api/rag/index/document/{documentId}/reprocess → Reprocess one document

DELETE /api/rag/index/document/{documentId} → Delete one document's RAG chunks and vectors

GET /api/rag/index/tasks → Recent index tasks; GET /api/rag/index/documents/status → Per-document index status

GET /api/rag/index/document/{documentId}/chunks → Persisted chunk text, vectorId, embedding profile, and index status

GET /api/rag/embedding/active → Active embedding profile, model, dimension, collection, and rebuild status

GET /api/rag/embedding/profiles → Available embedding profiles

POST /api/rag/embedding/profiles/{profileId}/test → Readiness probe

POST /api/rag/embedding/profiles/{profileId}/activate → Switch active embedding profile

GET/POST/PUT/DELETE /api/rag/kb → Knowledge-base management; POST /api/rag/kb/{kbId}/documents/upload → Upload and index source documents

POST /api/rag/access-request → Submit a RAG document access request

Gateway (Port 8080):

Routes: /api/user/**→user-service:8081; /api/tool/**→tool-agent:8083; /api/code/**→code-agent:8084; /api/rag/**→rag-agent:8085; /ws/**→task-service WebSocket endpoint

JwtAuthGlobalFilter (order=-100): Validates JWT on all non-whitelist paths; extracts claims → X-User-* headers; rate limiting: 100 req/s per IP (burst 200)

Appendix C: User Manual (Summary)

Access and Login. Navigate to <http://167.172.82.161> and log in with your assigned credentials. Default test accounts: - admin / 123456 (System Administrator, full access, clearance Level 3) - credit_mgr / 123456 (Credit Department Manager, clearance Level 2) - credit_staff / 123456 (Credit Department Staff, clearance Level 1) - compliance_mgr / 123456 (Compliance Department Manager, clearance Level 2) - compliance_staff / 123456 (Compliance Staff, clearance Level 1). New users can register via the Register page with an email-format username and a 6-digit email verification code.

Main Dashboard. The dashboard displays a welcome message with your username, a pending approval alert banner (for ROLE_ADMIN and ROLE_DEPT_ADMIN, showing the count of pending HITL requests with a “Review” button), and a statistics overview with four cards: total users, online users (WebSocket connections), today’s agent tasks processed, and pending approval requests.

Tool Agent (/app/tool). The left panel has three tabs: Meeting (room search with date and capacity filters, and booking), Schedule (conflict detection by attendee with a visual timeline grid), and Route (origin/destination input with map rendering via Amap, showing a route polyline and step-by-step guidance). The right panel is the AI Assistant Copilot, a global chat panel.

Code Agent (/app/code). A three-step workflow: (1) enter a natural language query or select a quick template, (2) review the generated SQL in a code editor with model information, and (3) execute and view results in a data table with pagination. The result table shows columns, row count, the generated SQL, and execution time. A security rejection message is displayed if the SQL fails any validation layer.

RAG Workbench (/app/rag). Users can ask policy and document questions in natural language. The page sends the query to the RAG Agent, displays the generated answer, and lists citations with document/chunk identifiers, snippets, and similarity scores. The sidebar shows RAG readiness, embedding status, accessible documents, and recent index tasks. Users can inspect the documents they are allowed to retrieve from, open cited documents in the document reader, and submit an access request when a required document is

outside their clearance scope. Administrators can rebuild the full RAG index or reprocess individual documents after document updates.

Documents (/app/dept-docs). Two tabs: System Manuals (global documents) and Department Assets (clearance-controlled). Click a document to open a Zen Reader with markdown rendering and an auto-generated table of contents. Documents beyond your clearance level show “Access Restricted” with an “Apply for Access” button that submits a RAG_APPLY notification. The document detail dialog also exposes RAG index status, chunk count, vectorId, embedding profile, and chunk text so administrators can verify whether a document has been correctly parsed and indexed.

Messages (/app/notification). A three-column layout: left sidebar with filters (All, Un-read, Pending Approvals, Sent), middle message list, and right detail panel. Type-specific renderers handle RAG_APPLY approvals (with approve/deny buttons), SQL_AUDIT records (with SQL code blocks), and BUG_REPORT feedback.

Register (/register). New users can create an account with an email-format username and a 6-digit email verification code. The registration form requires the user to enter their email address, request a verification code (sent to their inbox, valid for 5 minutes), set a password, and confirm the code. Server-side rate limiting enforces a 60-second cooldown between code requests and a maximum of 5 code requests per email address per day.

Administration (/app/admin/users, /app/admin/resources, /app/admin/my-dept). User Management provides a searchable table of all users with filters for department, clearance level, and role. Admins can assign roles, toggle account status, set clearance levels, and manage department membership. Resource Management lets admins manage meeting rooms and view all schedules. My Department (for ROLE_DEPT_ADMIN) shows members within their own department with the ability to view and manage membership.

Settings (/app/settings). Two configuration dialogs: Profile (update your display name through PUT /user/profile) and Security (change your password with current password verification, a new password must be at least 3 characters via PUT /user/password).

Declaration of Individual Contributions

In accordance with the University of Hong Kong's guidelines for group project reports, this section details the specific contributions of each team member.

Lin Chuanbin:

In the Agent Platform project, I was responsible for developing three core modules from scratch: the Tool Agent microservice, the User Service microservice, and the full-stack implementation of the initial front-end version, delivering a total of 30+ core files. Below are my specific responsibilities:

1. Tool Agent

The Tool Agent is the core intelligent agent of the system, built on Spring Boot, integrating the DeepSeek large language model to achieve natural language understanding and intent recognition. It covers five subsystems: AI intent routing, meeting room management, schedule conflict detection, intelligent route planning, and WebSocket real-time communication.

(1) System Architecture and AI Intent Recognition

I designed a layered architecture for the Tool Agent, clearly defining the Controller layer, Service layer, external service layer, and data access layer. The Controller layer provides a unified RESTful API entry point; the Service layer is responsible for core business functions such as intent routing, meeting room management, and schedule conflict detection; the external service layer encapsulates DeepSeek AI calls and the Gaode Map API; and the data layer implements database access through MyBatis-Plus.

The AI intent recognition module is the intelligent brain of the Tool Agent. I encapsulated the communication logic with the DeepSeek API, using a RESTful approach and supporting a dual-role dialogue mode of system prompts and user content. The system defines three intent types (route planning, meeting room search, and schedule conflict detection). After DeepSeek analyzes the intent, it automatically routes the request to the corresponding business processing method.

Considering the potential unavailability of AI services in a production environment, I implemented a robust fallback mechanism: when a DeepSeek call fails, the system returns

preset demo data based on the tool type, ensuring the front-end always displays correctly and the user experience is not affected by AI service failures.

(2) Meeting Room Management

The meeting room module contains two core data models: meeting room entities and schedule booking entities. I implemented time period overlap detection logic, returning complete data including availability status and a list of devices. The booking function first checks for conflicts; if the time period is already booked, it returns a failure.

For AI intelligent matching, the system calls DeepSeek to analyze the user's meeting room needs, extracting information such as date, number of people, and equipment. The returned results include an AI matching score and recommendation reasons, helping users quickly find the most suitable meeting room.

(3) Schedule Conflict Detection

Schedule data is stored in a Redis ordered set, using user ID as the key and timestamp as the score, achieving range queries with $O(\log N)$ time complexity. I designed three core interfaces: range query conflict detection, schedule data storage, and record deletion by event ID.

Timestamp calculation uses the East 8 time zone to ensure consistency with domestic business time.

(4) Intelligent Route Planning

I encapsulated the calls to the Gaode Map REST API, mainly using three interfaces: geocoding, POI search, and driving route planning. To address the address resolution success rate issue, I designed a three-level degradation strategy: first, attempt direct geocoding; if it fails and the address does not contain a city prefix, automatically append the "Beijing" prefix and retry (this was later modified); finally, use POI text search as a fallback, significantly improving the address resolution success rate.

The system formats the raw data returned by Gaode Maps in a user-friendly way: distance is automatically converted to "km/m" units, duration to "hours/minutes" units, and traffic conditions are automatically classified into three levels: "smooth/slow/congested" based on average speed.

(5) WebSocket Real-time Task Progress Push

The system provides a real-time task progress push service via WebSocket, supporting the establishment of independent connections by task ID. A concurrent and secure session management mechanism is used to manage all active connections, simulating a complete processing flow (parsing natural language → querying the database → AI model inference → organizing results), with progress updates pushed at fixed intervals for each stage. A thread pool is used to manage task execution, supporting a maximum of 10 concurrent tasks.

2. User Service

The User Service is the system's user authentication and permission management microservice, built on Spring Boot + Spring Security, using JWT to implement stateless identity authentication, supporting encrypted password storage and interface-level permission control. Yao Jiacheng and Xia Xin subsequently made some improvements and developments on this basis.

(1) System Architecture Design

The User Service adopts a layered architecture, including a Controller layer, a Service layer, a data access layer, and an entity/DTO layer. Global security control is implemented through Spring Security configuration, including CSRF disabling, stateless session management, allowing login interfaces, and authentication protection for other interfaces.

(2) JWT Authentication and Security Module

I encapsulated complete lifecycle management of JWTs, including token generation, parsing, and verification. The HMAC-SHA256 signature algorithm is used, with the key and expiration time injected from the configuration file for flexible configuration management. The generation method encodes the username and role information into the token; the parsing method verifies the token signature; the verification method catches all exceptions and returns a boolean value to ensure the security of the caller.

The security policy is configured with complete Spring Security protection: disabling CSRF (using JWT instead of Session in a front-end/back-end separation architecture), setting up stateless session management, allowing login interfaces, and requiring authentication for all other requests. Passwords are stored using BCrypt with one-way hash encryption.

(3) User Login and Permission Management

The login process implements a complete four-step security verification: querying users by username, checking user status, BCrypt password matching verification, generating a token, and encapsulating the response for return.

I designed standardized request and response DTOs, using annotations to implement parameter validation to ensure that usernames and passwords are not empty. I also defined a unified generic response encapsulation class containing three fields: status code, message, and data, providing static factory methods for success and failure to ensure consistent return formats across all microservices.

3. Front-end Development (Initial Version)

I was responsible for building the initial version of the front-end project and implementing the core pages, providing the basic framework for subsequent iterative development by team members (later, Yao Jiacheng and Xia Xin proposed many excellent new ideas and made many significant improvements). My front-end uses the Vue 3 + TypeScript + Vite technology stack, combined with the Element Plus UI component library and Pinia state management.

(1) Technology Architecture and Engineering

The project uses Vue 3's composable API to organize code logic, TypeScript for type safety, and Vite as the build tool for rapid development and hot updates. I configured path aliases to significantly improve code maintainability. I also configured API proxies and WebSocket proxies to solve cross-domain issues in the development environment. Element Plus uses on-demand plugin loading to import components as needed, reducing the package size.

(2) Core Page Implementation (Initial Version)

Login Page: Uses a centered card layout with a gradient background for a modern feel. A form component is used for username and password input, with validation rules ensuring input integrity. The login logic calls a state management method; upon successful login, the token is persisted and the user is redirected to the homepage.

Main Layout Page: Uses a classic sidebar + top navigation + main content area layout. The sidebar implements a multi-level menu, and the top navigation displays current user information and logout functionality. The main content area uses transition animations to switch pages.

The tool call page adopts a left-right split layout. The left side is the function operation area (three tabs: meeting room search, schedule conflict detection, and route planning), and the right side is the AI assistant panel (natural language input, WebSocket real-time progress, and AI analysis result display). The map component encapsulates the initialization, route drawing, origin and destination marking, and view adaptation functions of Gaode Map, and uses a dynamic script loading method to import the map API.

Note: The above front-end components and pages are the initial version. Subsequent versions were refactored and expanded by Yao Jiacheng and Xia Xin, including a multi-tab system, responsive mobile adaptation, a global Copilot component, and a unified UI design system—all production-grade improvements.

(3) State Management and HTTP Encapsulation

I designed a user state management module, defined using Pinia's Setup Store pattern, including responsive states such as tokens and user information, as well as Action methods such as login, logout, and retrieving user information. Upon successful login, the token is persisted to local storage; upon logout, the state is cleared and the user is redirected back to the login page. A task state management module was also designed, supporting adding tasks and updating task states.

The HTTP request encapsulation uses Axios to implement a unified request instance, configured with a relatively long timeout (to adapt to AI inference time), a basic path, and JSON content type. The request interceptor automatically reads the token and adds it to the request header. The response interceptor handles backend responses uniformly: returning data on success and displaying an error message on failure; unauthorized status triggers login expiration confirmation, and the user is automatically logged out and redirected to the login page after confirmation.

(4) WebSocket Client and Task Orchestration (Initial Version)

I encapsulated a WebSocket client class, providing core methods such as connect, send, close, and event listeners. It supports an automatic reconnection mechanism (up to 3 times, 2-second interval), and achieves loosely coupled message processing through an event listener pattern. A singleton pattern is used to ensure global sharing of the same connection.

Simultaneously, I wrote the initial gateway center orchestration logic, implementing static

Agent routing and WebSocket progress updates, providing the infrastructure for the subsequent implementation of asynchronous execution pipelines, failover, and retry mechanisms.

Yao Jiacheng:

I served as the backend architecture lead and, over the course of the project, took on full-stack development, system integration, security, and DevOps. Much of my work involved extending the initial implementations contributed by other members into production-grade features.

1. Backend

(1) RBAC authentication & authorization

Building on the initial user-service skeleton (basic JWT login with a single admin/user role) created by Lin Chuanbin, I extended the platform's security model into a multi-layer permission architecture:

- Designed five RBAC tables supporting three role types (admin, department admin, regular user) with fine-grained permissions pre-seeded in the database.
- Replaced the original single-role JWT filter with a custom `JwtAuthenticationFilter` that extracts roles and permissions from JWT claims and registers them in Spring Security, enabling declarative `@PreAuthorize` checks.
- Built admin and department-admin controllers covering user CRUD, role assignment, department assignment, clearance level management, and account lifecycle operations.
- Introduced multi-level department trees with automatic department-scoped data isolation in all data-access controllers.
- Implemented three-tier document clearance enforcement with department scoping, clearance level comparison, and HITL override via approved access requests.

(2) API gateway, session management & unified response format

I built the platform's unified API gateway layer and standardized the cross-service response format:

- Built a custom Spring Cloud Gateway global filter with route rules covering all microservices and WebSocket endpoints.
- Implemented single-session enforcement via Redis (new logins invalidate old sessions), sliding-window TTL renewal with polling endpoint exemption, and dynamic admin-configurable session timeout.
- Propagated user identity

through HTTP headers so downstream services operate without holding JWT_SECRET. - Defined the Result<T>generic wrapper standard, deployed identical DTOs across all four microservices, and wrote the frontend Axios interceptor with automatic unwrapping.

(3) Notification & HITL approval system

I independently designed and implemented the platform's notification and approval sub-system:

- Designed the notification table with threaded messaging support (parent_id/thread_id) and a five-state state machine spanning unread, read, pending, approved, and rejected statuses.
- Built the notification controller with endpoints for sending, filtered listing, read marking, approve/deny with audit trail, and threaded conversation view.

(4) Task center orchestration

Building on the initial static agent routing scaffold, I designed and implemented the complete task-service microservice as a centralized orchestration layer that tracks and manages all Agent workflows across the platform. When a user submits a request through any Agent, the task center persists it as a task record, queues it for asynchronous execution in a bounded thread pool, and pushes real-time progress updates to the frontend via WebSocket.

- Built the task-service microservice with a four-state lifecycle (INIT → RUNNING → SUCCESS / FAIL) and an async execution pipeline backed by a ThreadPoolExecutor.
- Implemented routing by task type (CODE / TOOL / RAG) to the corresponding Agent, with domain-meaningful progress milestones pushed at each stage.
- Designed fault-tolerance mechanisms including connection timeout, read timeout, up to 3 automatic retries with exponential backoff, and graceful queue overflow rejection via AbortPolicy to prevent system overload under high concurrency.

(5) Email verification system

I replaced the initial QQ SMTP implementation (originally contributed by Xia Xin) with a production-grade solution after identifying reliability and security limitations:

- Migrated from QQ SMTP to the Resend HTTP API with a redesigned HTML email template.
- Hardened security with brute-force protection against verification code guessing and a three-tier rate limiting system (per-email cooldown, per-IP hourly cap, per-email

daily cap).

(6) Copilot backend & AI provider unification

I built the backend for the global Copilot assistant, which generalized the AI chat panel originally confined to Lin Chuanbin's Tool Agent page into a cross-page, multi-Agent intelligent dispatcher that sits above all three agents as a unified natural language entry point. The Copilot's Python Flask intent classifier implements a three-tier classification pipeline — security blacklist, keyword greeting match, and LLM-based six-category classification (CODE, TOOL, RAG, CHAT, CLARIFY, SETTINGS) — complemented by multi-turn clarification, cross-page result delivery via browser custom events, and WebSocket-based progress streaming for long-running agent calls. I unified all AI provider configurations across the platform into a single admin-managed configuration center with AES-256 encryption, eliminating the fragmentation that existed when each agent managed its own LLM settings independently.

(7) Document management & MinIO storage

I worked with Huang Siyuan to build the document storage and management backend, which serves both the document library and her RAG Agent:

- Built a MinIO storage service with auto bucket creation, admin-configurable upload size limits, and support for PDF, DOCX, PPT, and Markdown files.
- Extended the document table with file metadata and parse status columns to support RAG document parsing.
- Implemented document access request approval routing to department managers with fallback to system admin.

2. Frontend

(1) Layout, navigation & multi-tab system

I rebuilt the frontend shell from the initial scaffold created by Lin Chuanbin into a production-grade application framework:

- Rebuilt the main layout with a collapsible sidebar, multi-tab top bar, responsive mobile layout with off-screen drawer, and role-based dynamic menu visibility.
- Built a browser-style multi-tab system that preserves page state when users switch between tabs — form inputs, filters, and scroll positions remain intact — preventing task interruption, a common issue in enterprise workflows.
- Implemented a five-layer global navigation guard

covering silent refresh recovery, authentication redirect, bounce-back, single-role check, and multi-role check.

(2) UI design system

I established a minimalist visual design language across the entire platform, replacing the default Element Plus styling inherited from the initial scaffold:

- Globally replaced the default blue theme with a grayscale color system, removing accent strips and unifying border-radius, sizing, and spacing.
- Redesigned notifications with macOS-style frosted glass and custom cubic-bezier slide-in animation.

(3) Tool Agent UI rebuild

I redesigned all three Tool Agent interfaces originally built by Lin Chuanbin, splitting the monolithic 600-line file into independent sub-components and removing the fixed AI chat panel in favor of an inline interaction model:

- Meeting Agent: rebuilt search as a horizontal inline bar with a responsive card grid, SVG icons, equipment tags, and a booking dialog.
- Schedule Agent: unified two separate forms into a single filter panel and replaced manual user ID input with a filterable multi-select dropdown.
- Route Agent: redesigned the map from conditional rendering (hidden when no results) to a persistent full-width display. Added a floating control panel and clickable route segments that show detailed navigation steps.

3. DevOps

(1) Containerization & deployment

I set up the full deployment pipeline for the platform, bringing all Agent services — Zhang Zhaoming's Code Agent, Huang Siyuan's RAG Agent, and Lin Chuanbin's initial Tool Agent — onto a single DigitalOcean Droplet:

- Wrote the docker-compose.yml that starts all services with proper startup order, health checks, and data persistence.
- Created Dockerfiles for each Spring Boot service.
- Deployed a DigitalOcean Droplet. To keep the Maven build from running out of memory on a small instance, I compiled the JARs locally and copied them over.
- Set up Nginx as the only public-facing entry point, handling SPA routing, API/WebSocket proxying, and static file caching.
- Registered the domain bankagent.online through Namecheap and configured Let's Encrypt for HTTPS access.

Huang Siyuan:

I served as the RAG Agent lead and independently designed and implemented the platform's enterprise knowledge retrieval agent. My work covered the full RAG Agent lifecycle, including backend microservice development, vector database integration, embedding service design, document indexing, permission-aware retrieval, knowledge base support, frontend interface alignment, local environment setup, debugging, and end-to-end testing. After completing the RAG backend and database design, I coordinated with Yao Jiacheng on gateway routing, user identity propagation, document metadata compatibility, shared database changes, Docker Compose dependencies, and deployment configuration. For frontend-facing RAG functions, I aligned API fields, page behavior, citation display, document status, and administrator settings with Xia Xin and Yao Jiacheng.

1. Backend

(1) RAG microservice and API design

I independently built the rag-agent microservice as a standalone Spring Boot service running on port 8085 and integrated it into the platform through the Gateway route `/api/rag/**`. I implemented the main controller, service, DTO, entity, mapper, and configuration layers, and exposed the core RAG APIs including `/rag/query`, `/rag/health`, `/rag/health/embedding`, `/rag/index/rebuild`, `/rag/index/documents/status`, `/rag/embedding/profiles`, and `/rag/embedding/active`. This made the RAG Agent a dedicated intelligent agent module in the platform, rather than a separate experimental component.

(2) RAG retrieval pipeline

I implemented the complete RAG retrieval workflow. The service receives a user question, reads user identity from gateway-propagated headers, generates a query embedding, searches Milvus for candidate chunks, applies permission verification, assembles retrieval context, and returns a citation-based answer. The response includes trace ID, status, answer, citations, chunks, retrieved document IDs, blocked document IDs, topK, latency, and diagnostic message, so that both users and developers can inspect how an answer was produced.

(3) RAG database and metadata model

I designed and implemented the RAG metadata model around `rag_document_chunk`, `rag_query_log`, `sys_document`, and `sys_config`. I added and maintained fields such as `vector_id`, `embedding_profile`, `embedding_model`, `vector_collection`, `content_hash`, `chunk_index`, `index_status`, and `last_indexed_at`. These fields allow each retrieved chunk to be traced back to its source document, embedding model, active profile, and Milvus collection, and they also support index rebuilds, profile switching, and document index status inspection.

(4) Permission-aware retrieval

I aligned the RAG Agent with the platform's RBAC, department, and clearance-level security model. The service uses identity headers such as `X-User-Name` and `X-User-Roles` propagated by the Gateway, then checks the user's department and clearance level before allowing document content into the retrieval context. I also implemented blocked-document tracking and controlled `NO_CONTEXT` responses, preventing unauthorized or cross-department documents from appearing in generated answers and citations.

2. Vector Store and Embedding

(1) Milvus integration

I integrated Milvus as the vector database for the RAG Agent and configured it with etcd and MinIO in Docker Compose. I implemented collection initialization, vector insertion, vector search, vector ID generation, and similarity retrieval using COSINE distance. I verified that Milvus search results could be matched back to MySQL chunk records and source documents.

(2) Local embedding worker

I built and tested a local Python `rag-worker` using the BGE-M3 embedding model. The worker exposes an HTTP `/embed` endpoint and returns 1024-dimensional embeddings. The Java RAG Agent calls this worker through HTTP, keeping the Spring Boot service independent from Python model runtime details. I also handled local worker setup, including virtual environment creation, dependency installation, model download, Hugging Face cache behavior, startup delay, health checks, and embedding dimension verification.

(3) Qwen cloud embedding API support

I extended the embedding layer to support Qwen `text-embedding-v4` through a cloud

API. I added configuration for provider type, endpoint, API key, model name, vector dimension, timeout, and collection name. This allows the RAG Agent to support both local BGE-M3 embeddings and cloud-based Qwen embeddings.

(4) Embedding profile isolation

I designed the embedding profile mechanism to keep different embedding models isolated. Each profile has its own provider, model, dimension, endpoint, active status, index status, and Milvus collection. For example, `local-bge-m3` stores vectors in `rag_document_chunks_bge_m3`, while `qwen-v4` stores vectors in `rag_document_chunks_qwen_v4_1024`. I implemented APIs for listing profiles, checking the active profile, testing providers, activating profiles, and rebuilding the index after switching profiles. This avoids mixing incompatible vector spaces and keeps retrieval results technically valid.

3. Knowledge Base and Document Management

(1) Document storage collaboration

I worked with Yao Jiacheng on aligning the document management backend with the RAG Agent. The shared design uses MinIO for uploaded files such as PDF, DOCX, PPT, Markdown, and text files, while MySQL stores metadata such as file type, storage object key, parse status, department ID, security level, content hash, and index status. My work focused on defining and using the metadata needed by RAG so documents can be parsed, indexed, retrieved, and permission-filtered through the same knowledge base source.

(2) Document chunking and indexing

I implemented and tested the document chunking and indexing workflow. The RAG Agent reads document content, splits it into chunks, assigns chunk indexes, records content hashes, generates embeddings, inserts vectors into Milvus, and saves chunk metadata into MySQL. This enables chunk-level retrieval instead of whole-document matching, improving answer relevance and supporting citation display.

(3) Index rebuild workflow

I implemented the active-profile index rebuild workflow. When documents are added or changed, or when the administrator switches embedding providers, the RAG Agent can rebuild the index so that MySQL chunk metadata and Milvus vectors match the current embedding profile. I verified the rebuild process through `/rag/health`,

/rag/index/documents/status, direct MySQL checks, and actual RAG queries.

4. Frontend and Integration

(1) RAG frontend interface alignment

I coordinated with Xia Xin and Yao Jiacheng on frontend-facing RAG functions. I defined the backend response fields and interaction requirements needed for RAG answer display, citation panels, retrieved chunks, document IDs, similarity scores, snippets, blocked documents, index status, and embedding profile status. This helped the frontend present RAG answers with supporting evidence instead of only generated text.

(2) Administrator embedding provider switch

I proposed and helped design the administrator-side embedding provider switch. The frontend can expose local BGE-M3 and Qwen text-embedding-v4 as selectable options, while sensitive information such as API keys remains on the backend side. I also clarified the rule that switching providers requires rebuilding the index because each embedding profile uses a separate vector collection.

(3) Cross-service integration

After completing the RAG backend and database design, I coordinated with Yao Jiacheng to integrate RAG with the shared backend architecture. This included Gateway routing, propagated user headers, document metadata compatibility, database patching, Docker Compose dependencies, and deployment configuration. These integration steps ensured that the RAG Agent could work with the existing authentication system, document library, frontend pages, and deployment pipeline.

5. Testing, Debugging, and Documentation

(1) Local environment and service verification

I configured and tested the local RAG development environment, including MySQL, Redis, Milvus, etcd, MinIO, the RAG Agent, and the Python embedding worker. I verified Docker container health, Milvus ports, RAG service health, embedding worker health, database schema patches, and environment variables for vector storage, embedding providers, LLM configuration, and document storage.

(2) Backend debugging

I debugged integration issues across the RAG stack, including Docker daemon startup failures, port conflicts, Maven dependency download failures, MySQL schema mismatches, Milvus connection problems, embedding worker startup delay, Hugging Face model download problems, user identity mismatch, missing users, empty retrieval context, failed embedding calls, and index status inconsistency. These debugging tasks were necessary to make the RAG Agent run reliably inside the full platform.

(3) End-to-end RAG testing

I tested the complete RAG workflow through `/rag/health`, `/rag/health/embedding`, `/rag/index/rebuild`, `/rag/index/documents/status`, `/rag/embedding/active`, and `/rag/query`. I also checked MySQL directly to confirm that chunk metadata, vector IDs, embedding profiles, embedding models, and collection names were stored correctly. After rebuilding the active profile, I confirmed that RAG queries returned successful results with citations and permission-safe retrieved chunks.

(4) Documentation and team communication

I wrote and maintained RAG-related documentation, including progress reports, interface alignment notes, environment variable explanations, startup steps, embedding provider configuration, local worker setup, and index rebuild guidance. These documents helped other team members understand how to run, test, and integrate the RAG Agent.

Overall, my contribution was to design, implement, test, and integrate the complete RAG Agent for the enterprise multi-agent platform. The final RAG Agent includes a standalone Spring Boot microservice, Milvus vector retrieval, local and cloud embedding support, embedding profile isolation, document indexing, permission-aware retrieval, citation-based responses, knowledge base management support, frontend interface alignment, deployment configuration, debugging, documentation, and end-to-end verification.

Xia Xin:

I contributed across backend security, frontend internationalization, documentation standardization, authentication, multilingual support, and visual communication of system architecture.

1. Backend

(1) Authentication

I built the platform's initial Spring Security layer with stateless JWT tokens (HMAC-SHA256) and BCrypt password encoding to separate admin and regular user access. I configured the security filter chain to expose only the login and registration endpoints publicly while requiring authentication for all other routes, and defined the foundational Result<T> response wrapper and DTOs to ensure consistent error handling and data exchange across the codebase. This part was upgraded by Yao Jiacheng.

(2) Email Verification Integration Building on the existing team codebase, I integrated the QQ Mail SMTP service to deliver verification codes during registration. I stored codes in Redis with a 5-minute TTL, enforced a 60-second per-email send rate limit to prevent abuse, and wired the code verification step directly into the registration flow, completing the end-to-end sign-up process. This part has been upgraded into Resend HTTP API by Yao Jiacheng.

2. Frontend & Internationalization

(1) Multi-Language i18n Support I adapted all business and UI components to support three languages (Simplified Chinese, Traditional Chinese, and English) with dynamic switching through Vue3 and Element Plus i18n. This required systematically reorganizing component-level text, validation messages, and document metadata into structured locale files, ensuring the entire frontend remained maintainable and language-agnostic. Also, I designed a initial version of Code Agent's frontend, which has been upgraded into a better version by Yao Jiacheng.

3. Documentation & Visualization Standardization

(1) System Diagrams Redesign I designed the first version of all figures, and after team discussion, I redrew the original dense, small-font diagrams into clean, wide-format vector graphics with a unified visual style. Every diagram maintained readability and visual hierarchy, ensuring clear communication of complex architectural concepts. A total of 12 figures were created and refined.

(2) Final Report & Table Standardization

I was responsible for synchronizing the final report with the latest codebase. I converted all seven data tables into the three-line table format (1.5 pt top and bottom rules, a 0.75 pt rule below the header, no vertical or interior horizontal lines) and suggested moving long pseudocode blocks to standalone tables—both to maintain formatting consistency and to

communicate algorithms more clearly to readers.

Zhang Zhaoming:

I designed and implemented the Code Agent module — the natural-language-to-SQL engine that translates user questions into MySQL queries, validates them through a five-layer security whitelist, and executes them safely against the database. The module comprises a dedicated Spring Boot microservice and a Python LLM inference bridge. I also conducted model-level experimentation, fine-tuning two T5 variants before converging on the final LLM API-based approach.

1. Code Agent Microservice Architecture

I built the code-agent microservice (port 8084) from scratch as a standalone Spring Boot 3.2 service.

Designed the service with a clean separation of concerns: CodeAgentController (REST layer, 6 endpoints), CodeGenerationService (interface with pluggable implementations), SqlValidationService (five-layer whitelist engine), SqlExecutionService (JdbcTemplate-based safe execution), MetadataCacheService (information_schema → Redis), and MetadataCacheManager (lifecycle management).

Implemented the OnnxCodeGenerationService as an HTTP proxy to the Python LLM inference server (port 8090), using java.net.http.HttpClient with 10-second connect timeout and 60-second response timeout. The contract is POST question → returns sql, method JSON.

Built the CodeAgentProperties configuration class with @ConfigurationProperties(prefix = "code-agent"), supporting three config groups (ONNX inference, SQL whitelist, metadata cache) all overridable via environment variables for seamless Dev/Prod switching.

2. Five-Layer SQL Whitelist Validation Engine

I designed and implemented a defense-in-depth SQL validation pipeline (SqlValidationService) that blocks injection and unauthorized queries at five sequential layers, each with immediate rejection on failure:

Layer 1 — Operation Type: Enforces SELECT-only. Any non-SELECT statement is rejected before further processing, preventing write operations at the earliest stage.

Layer 2 — Forbidden Keywords: Scans SQL with word-boundary regular expressions (`\bDROP\b`, `\bDELETE\b`, `\bINSERT\b`, `\bUPDATE\b`, `\bALTER\b`, `\bTRUNCATE\b`, `\bCREATE\b`, `\bEXEC\b`, `\bUNION\b`, `\bINTO\b`, `\bLOAD_ FILE\b`, `\bOUTFILE\b`, `\bDUMPFILE\b`). The `\b` boundary prevents false positives — for example, `customer_no` is not flagged for containing the substring `IN`.

Layer 3 — Table Whitelist: Extracts all tables referenced in `FROM` and `JOIN` clauses via regex and validates each against the cached `information_schema` table list. Unknown tables are rejected with the allowed set shown in the error message.

Layer 4 — Column Whitelist: For non-`SELECT *` queries, extracts every column from the `SELECT` clause and validates each against the actual column definitions of the referenced tables — supporting `JOIN` queries by merging all referenced tables' column sets. Aggregate functions and expressions are intelligently skipped.

Layer 5 — Complexity Limits: Caps queries at a maximum of 3 joined tables and 10 conditions (counting `AND/OR` occurrences + 1 base). This prevents resource exhaustion from overly complex LLM-generated queries.

All layer parameters are configurable through `code-agent.whitelist.*` in `application.yml`.

3. Metadata Caching & Execution Infrastructure

I built the metadata caching layer and safe SQL execution service:

Implemented `MetadataCacheService` that queries MySQL `information_schema` (using JDBC directly, not MyBatis) to extract table names, column names, data types, and comments — caching them in Redis as a JSON Hash map keyed by table name with a 1-hour TTL. When Redis is unavailable, the service falls back to direct MySQL queries, ensuring zero-downtime operation.

Built `MetadataCacheManager` with an `@EventListener(ApplicationReadyEvent.class)` for startup cache warm-up and a `@Scheduled(fixedRate = 30 * 60 * 1000)` for periodic refresh every 30 minutes.

Implemented `SqlExecutionService` with defense-in-depth: before executing any SQL, it re-validates through the full five-layer pipeline, catching edge cases where SQL might have been tampered with between generation and execution. Only then does it run via `JdbcTemplate.queryForList(sql)` and return results with column names and data rows.

Built the `generateAndExecute` one-shot method that chains generation → validation → execution into a single `FullResult` response.

4. REST API Endpoints

I designed and implemented all six REST endpoints for the Code Agent, centered around the Human-in-the-Loop (HITL) pattern that separates generation from execution:

`POST /code/query` is the one-shot endpoint: it accepts a natural language question, calls the LLM inference service to generate SQL, runs the five-layer validation, executes the query, and returns the full result including column headers, data rows, row count, and elapsed time.

`POST /code/generate` handles SQL generation only — it takes a question and returns the LLM-generated SQL with inference metadata, without executing anything. This enables the frontend to display the generated SQL for human review before it touches the database.

`POST /code/execute` accepts a reviewed (and potentially human-edited) SQL string, re-validates it through the full whitelist pipeline, and executes it — completing the supervised HITL workflow.

`GET /code/metadata` returns the currently cached table metadata from Redis or MySQL fallback.

`POST /code/metadata/refresh` forces an immediate refresh of the metadata cache from `information_schema`.

`GET /code/health` returns the service status and active inference method for operational monitoring.

5. Model-Level Experimentation: Fine-Tuned T5 vs. LLM API

Before settling on the final architecture, I conducted practical experiments comparing local fine-tuned models against cloud LLM APIs for the text-to-SQL task:

Fine-tuned two T5 variants on our banking-domain dataset, deploying them via ONNX Runtime for local inference. The goal was to evaluate whether a small, domain-specific model could match general-purpose LLMs on our constrained schema.

Through systematic comparison, I found that the fine-tuned T5 models consistently underperformed the LLM API approach — even with careful training, the smaller models

struggled with schema grounding, complex JOIN logic, and edge cases that a general-purpose LLM handled naturally when given well-structured metadata in the prompt.

This led to the key architectural decision: rather than investing further in local model optimization, I adopted the LLM API + rich metadata prompt strategy, where the cached `information_schema` context (table names, column definitions, data types, example values) is injected into every generation request. This approach proved both more accurate and more maintainable, as schema changes require no model retraining — only a metadata cache refresh.

6. Test Dataset & Banking Domain

I authored the `banking_data.sql` test dataset with three relational tables: `bank_customer` (5 rows, risk levels LOW, MEDIUM, and HIGH), `bank_account` (7 rows, account types SAVINGS, CHECKING, and FIXED), and `bank_transaction` (10 rows, transaction types DEPOSIT, WITHDRAW, and TRANSFER). These cover diverse NL2SQL scenarios from simple lookups to multi-table JOIN aggregations, providing a realistic testbed for the generation, validation, and execution pipeline.